EW-RIE*: Làm sạch ma trận tương quan có trọng số mũ bằng lý thuyết ma trận ngẫu nhiên

Ứng dụng: danh mục phương sai nhỏ nhất trên HOSE — **bản v3 gọn trên khung của thầy** (dữ liệu HOSE 2015–11/2026, gồm mã hủy niêm yết; T1–T5)

Câu hỏi nghiên cứu. Ở thị trường mới nổi (N cổ phiếu lớn so với số phiên T, biến động thay đổi theo chế độ, cấm bán khống), bộ ước lượng ma trận hiệp phương sai nào cho danh mục rủi ro thấp nhất ngoài mẫu?

Phương pháp đề xuất: EW-RIE*

1. Ước lượng tương quan có trọng số mũ (half-life *h*): $E = \sum_t w_t x_t x_t^\top = C^{1/2} W C^{1/2}$, với $W = \sum_t w_t y_t y_t^\top$ là nhiễu.
2. R-transform của nhiễu: $R_W(g) = \sum_t \dfrac{w_t}{1 - N w_t g}$.
3. Phép thế nhân (BBP 2016): $\xi_i = \lambda_i \dfrac{\operatorname{Im}\gamma(z_i)}{\operatorname{Im} t_E(z_i)}$, $z_i = \lambda_i - i\eta$, $t_E = z g_E(z) - 1$, $\gamma\, R_W(\gamma) = t_E$. Trọng số đều ⇒ trùng RIE cổ điển.
4. Chọn đúng nhánh nghiệm (tiếp tục giải tích), xử lý ngoại lai trên trục thực, hiệu chỉnh N hữu hạn.

Đối chứng: ma trận mẫu, Ledoit–Wolf (2004), NL-LW (2020), MP-clip, RIE (BBP 2016/17), EW-RIE (q_{eff}), EW-NL-LW (**sửa cho N > T_eff**), **EW-WeSpeR (Oriol 2024)**, VS-NL-LW, VS-RIE; T3 thêm 1/N, PCA, DCC-GARCH.

Kết quả chính (§4: HOSE 12/2016–11/2026, 118 kỳ OOS, N=100, T=250, h=60, REB=21, lọc đứng giá ZERO_MAX=20%; Vol ngoài mẫu %/năm):

	Bán khống	**Chỉ mua**
NL-LW	13,10	13,72
EW-NL-LW	12,72	13,12
EW-RIE (q_eff)	14,30	14,76
EW-WeSpeR	12,57	**13,05**
EW-RIE*	**12,56**	13,07
VS-NL-LW	*12,49*	*12,93*

Đọc nhanh T1–T5

- **T1 (dữ liệu):** 105 mã rời HOSE (86 mã có giá, 19 mã thiếu giá); 15,3% phiên đứng giá ⇒ lọc `ZERO_MAX`.
- **T2 (độ vững):** EW-RIE* có Vol thấp nhất trong 3 phương pháp ở 6/6 tổ hợp REB × chế độ; sau phí NL-LW hơn ở REB = 21/63 (turnover thấp hơn), EW-RIE* hơn ở REB = 5. Không lọc đứng giá thì rủi ro bị đánh giá thấp ≈ 1,5 pp.
- **T3 (đối chứng, kiểm định):** EW-RIE* thấp hơn có ý nghĩa (Holm) so với NL-LW, EW-NL-LW, PCA-auto, 1/N; **không khác EW-WeSpeR** (p ≥ 0,47). Sharpe không khác biệt có ý nghĩa.
- **T4 (lý thuyết):** nhánh nghiệm, điều kiện ngoại lai (ngưỡng θ_c = 1,98 với h = 60) và hiệu chỉnh N hữu hạn được kiểm chứng số; rà soát tính mới: WeSpeR (Oriol 2024) giải cùng bài toán ⇒ cần xác nhận toàn văn.
- **T5 (VS):** VS giảm Vol ở 4/4 cặp (−0,6 đến −0,8 pp), lợi ích dồn vào COVID–2021; không làm sạch tương quan mà ước lượng biến động riêng lẻ kịp thay đổi.

Tài liệu chính: Bun, Bouchaud & Potters (2016) *IEEE Trans. Inf. Theory* 62(12); (2017) *Physics Reports* 666; Ledoit & Wolf (2020) *Ann. Statist.*; Oriol (2024) *WeSpeR*, arXiv:2410.14413; Laloux et al. (1999) *PRL*; Jagannathan & Ma (2003) *J. Finance*.

Phân công nhóm

Mã	Việc	Đầu ra	Mục trong notebook
T1 ✔	Dữ liệu 2020–2025 (vnstock/nguồn trả phí) + **mã đã hủy niêm yết** để loại survivorship bias — *đã làm: HOSE 2015–11/2026, gồm mã hủy niêm yết*	`prices.csv`, `values.csv`	§2, ô T1
T2 ✔	Độ vững: chi phí GD thực tế (phí + thuế 0,1%), tần suất tái cân bằng 5/21/63 phiên, xử lý phiên không giao dịch (~18%)	Bảng độ vững	§6, ô T2
T3 ✔	Đối chứng & kiểm định chuẩn: danh mục 1/N, mô hình nhân tố, DCC-GARCH; kiểm định chênh lệch phương sai Ledoit–Wolf (2011, HAC), Sharpe (Jobson–Korkie/Memmel)	Bảng kiểm định	§6, ô T3
T4 ✔	Lý thuyết: chứng minh tính đúng của việc chọn nhánh, điều kiện ngoại lai, hiệu chỉnh N hữu hạn; **rà soát tính mới** (đọc toàn văn BBP 2016 §III và Phys. Rep. 2017)	Mục Methodology (LaTeX)	—
T5 ✔	Vì sao VS giúp trên HOSE nhưng không trong GARCH? (biến động riêng lẻ, tương quan đổi theo chế độ...)	Ghi chú + thí nghiệm	§6, ô T5
T6	Mở rộng: N = 50/150, cửa sổ T, VN30/HNX	Bảng phụ lục	§5 (đổi CONFIG)
T7	Viết bài, nhắm tạp chí (kiểm tra hạng Q trên Scimago)	Bản thảo	—

Quy ước: mỗi người làm trên bản sao notebook (`File → Save a copy in Drive`), ghi kết quả vào thư mục chung. Đổi tham số ở ô CONFIG, không sửa thư viện ở §1 trừ khi đã thống nhất.

Khác với file của thầy (ngắn gọn): các ô của thầy giữ nguyên từng chữ; ô thêm vào đều có dấu ▶ **v3**. Chỉ ô CONFIG đổi (thêm `ZERO_MAX`, `DATA_SOURCE="csv"`, `DATA_DIR`). Việc thêm: (1) dữ liệu `data_hose/` 2015 → nay, có mã hủy niêm yết; (2) lọc mã đứng giá `ZERO_MAX = 20%`; (3) sửa EW-NL-LW khi N > T_eff; (4) đối chứng EW-WeSpeR (Oriol 2024); (5) điền T1–T5 ở §2 và §6.

0. Cài đặt & cấu hình

In [1]:
```
import os, sys, subprocess, time, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.covariance import LedoitWolf
warnings.filterwarnings("ignore")

IN_COLAB = "google.colab" in sys.modules
print("Colab:", IN_COLAB, "| numpy", np.__version__, "| pandas",
pd.__version__)
```

```
Colab: False | numpy 2.5.3 | pandas 3.0.5
```

In [2]:
```
# ======================== CONFIG ========================
CONFIG = dict(
    N=100,            # số mã thanh khoản nhất mỗi kỳ
    WINDOW=250,       # cửa sổ ước lượng (phiên)
    REB=21,           # tái cân bằng mỗi REB phiên
    COST=0.0025,      # chi phí 1 chiều (giả định, T2 cần chỉnh)
    HL_EW=60,         # half-life trọng số mũ
    HL_VOL=20,        # half-life chuẩn hóa biến động (VS)
    MAX_ABS_RET=0.15, # |r| > ngưỡng coi là lỗi dữ liệu (biên độ HOSE ±7%)
    ZERO_MAX=0.20,    # v3: loại mã đứng giá — tỷ lệ phiên r = 0 trong cửa sổ quá khứ > ZERO_MAX (1.0 = không lọc, như bản thảo)
    QUICK=False,      # True: chạy nhanh với ít bộ ước lượng
)
DATA_SOURCE = "csv"      # v3: "csv" = data_hose/ (2015–11/2026, có mã huỷ niêm yết) | "github" (2006–3/2020, cũ) | "vnstock"
# v3: thư mục data_hose/ có sẵn cạnh notebook thì dùng; không có (chạy trên web/Colab) thì ô T1.0 tự tải từ GitHub
DATA_DIR = next((os.path.abspath(d) for d in ("../data_hose", "./data_hose", "data_hose") if os.path.exists(f"{d}/prices.csv")), None)
WORKDIR = os.environ.get("RMT_WORKDIR", "/content/rmt" if IN_COLAB else "./rmt_work")
os.makedirs(WORKDIR, exist_ok=True)
CONFIG
```

Out[2]:
```
{'N': 100,
 'WINDOW': 250,
 'REB': 21,
 'COST': 0.0025,
 'HL_EW': 60,
 'HL_VOL': 20,
 'MAX_ABS_RET': 0.15,
 'ZERO_MAX': 0.2,
 'QUICK': False}
```

1. Thư viện: các bộ ước lượng

Không cần sửa. Mỗi bộ ước lượng nhận ma trận lợi suất `X` (T×N) và trả về ma trận hiệp phương sai N×N.

In [3]:

```
# ---------- tiện ích ----------
def cov_to_corr(S):
    sd = np.sqrt(np.diag(S)); return S / np.outer(sd, sd), sd

def renorm(C):
    d = np.sqrt(np.diag(C)); return C / np.outer(d, d)

# ---------- đối chứng cổ điển ----------
def est_sample(X): return np.cov(X, rowvar=False)

def est_lw(X): return LedoitWolf().fit(X).covariance_

def nl_lw_S(S, n):
    # Ledoit & Wolf (2020), co phi tuyến giải tích; S: hiệp phương sai, n: cỡ mẫu (hiệu dụng)
    p = len(S); lam, U = np.linalg.eigh(S); lam = np.maximum(lam, 1e-12)
    L = np.tile(lam, (p, 1)).T; H = n ** (-1 / 3) * L.T; x = (L - L.T) / H
    ft = (3 / 4 / np.sqrt(5)) * np.mean(np.maximum(1 - x**2 / 5, 0) / H, axis=1)
    with np.errstate(divide="ignore", invalid="ignore"):
        Hf = (-3/10/np.pi) * x + (3/4/np.sqrt(5)/np.pi) * (1 - x**2/5) * np.log(np.abs((np.sqrt(5) - x) / (np.sqrt(5) + x)))
    e = np.isclose(np.abs(x), np.sqrt(5)); Hf[e] = (-3/10/np.pi) * x[e]
    Ht = np.mean(Hf / H, axis=1); c = p / n
    d = lam / ((np.pi*c*lam*ft)**2 + (1 - c - np.pi*c*lam*Ht)**2)
    return (U * d) @ U.T

def est_nllw(X):
    Xc = X - X.mean(0); n = len(X) - 1
    return nl_lw_S(Xc.T @ Xc / n, n)

def mp_corr(C, q):
    lam, V = np.linalg.eigh(C); N = len(C)
    lp = (1 - lam[-1] / N) * (1 + np.sqrt(q))**2
    noise = lam < lp; lam = lam.copy()
    if noise.any(): lam[noise] = lam[noise].mean()
    return renorm((V * lam) @ V.T)

def rie_corr(C, q):
    # RIE Bun–Bouchaud–Potters (2016) + hiệu chỉnh N hữu hạn (2017, §8). Đây là bản GỐC (đối chứng).
    N = len(C); lam, V = np.linalg.eigh(C); lam = np.maximum(lam, 1e-10)
    z = lam - 1j / np.sqrt(N); g = (1 / (z[:, None] - lam[None, :])).mean(1)
    xi = lam / np.abs(1 - q + q*z*g)**2
    s2 = lam[0] / (1 - np.sqrt(q))**2 if q < 1 else lam.mean()
    lp, lm = s2 * (1 + np.sqrt(q))**2, s2 * (1 - np.sqrt(q))**2
    gmp = (z + s2*(q - 1) - np.sqrt(z - lm) * np.sqrt(z - lp)) / (2*q*z*s2)
    xi = xi * np.maximum(1, s2 * np.abs(1 - q + q*z*gmp)**2 / lam)
    xi *= N / xi.sum(); return renorm((V * xi) @ V.T)

def _corr_wrap(cleaner):
    def est(X):
        T, N = X.shape; C, sd = cov_to_corr(np.cov(X, rowvar=False))
        return cleaner(C, N / T) * np.outer(sd, sd)
    return est
est_mp, est_rie = _corr_wrap(mp_corr), _corr_wrap(rie_corr)
```

In [4]:

```python
# ---------- EW (trọng số mũ) và VS (chuẩn hóa biến động) ----------
def ew_weights(T, hl):
    w = 0.5 ** ((T - 1 - np.arange(T)) / hl); return w / w.sum()

def ew_corr(X, w):
    Xc = X - w @ X; S = (Xc * w[:, None]).T @ Xc
    sd = np.sqrt(np.diag(S)); return S / np.outer(sd, sd), sd

def make_ew(cleaner, hl):
    # cleaner(C, w) -> tương quan đã làm sạch
    def est(X):
        w = ew_weights(len(X), hl); C, sd = ew_corr(X, w)
        return cleaner(C, w) * np.outer(sd, sd)
    return est

def ewma_standardize(X, hl):
    lam = 0.5 ** (1 / hl); s2 = X[:hl].var(0) + 1e-10; Z = np.empty_like(X)
    for t in range(len(X)):
        Z[t] = X[t] / np.sqrt(s2); s2 = lam * s2 + (1 - lam) * X[t]**2
    return Z, np.sqrt(s2)

def make_vs(corr_cleaner, hl_vol):
    # corr_cleaner(Z) -> ma trận tương quan đã làm sạch của lợi suất chuẩn hóa Z
    def est(X):
        Z, sig = ewma_standardize(X, hl_vol); Z = Z[hl_vol:]
        return corr_cleaner(Z) * np.outer(sig, sig)
    return est
```

In [5]:

```
# ---------- EW-RIE*: RIE chính xác cho trọng sô´ mũ (phương pháp đê` xuâ´t)
----------
_STATS = {"fail": 0, "total": 0}; _NULL_CACHE = {}

def _F(g, t, a, w):
    d = 1 - a[None, :] * g[:, None]; R = (w[None, :] / d).sum(1)
    return g * R - t, R, (w[None, :] * a[None, :] / d**2).sum(1)

def _solve_gamma(t, a, w, g0, iters=30):
    # Newton có tìm kiê´m lùi cho  γ·R_W(γ) = t  (vector phức), a = N·w
    g = g0.copy(); f, R, dR = _F(g, t, a, w)
    for _ in range(iters):
        step = f / (R + g * dR); lam_ = np.ones(len(g))
        for _ in range(30):
            gn = g - lam_ * step; fn, Rn, dRn = _F(gn, t, a, w)
            bad = ~(np.abs(fn) < np.abs(f)) & (np.abs(f) > 1e-14)
            if not bad.any(): break
            lam_[bad] *= 0.5
        keep = bad | ~np.isfinite(fn)
        gn[keep], fn[keep], Rn[keep], dRn[keep] = g[keep], f[keep], R[keep],
dR[keep]
        g, f, R, dR = gn, fn, Rn, dRn
        if np.max(np.abs(f)) < 1e-13: break
    return g, np.abs(f)

def _continuation(t, a, w, steps=60, strict=True):
    # Nhánh chính: γ ≈ t khi t→0, đi theo tia s·t (s: 0→1)
    g = 1e-3 * t
    for s in np.geomspace(1e-3, 1.0, steps):
        g, res = _solve_gamma(s * t, a, w, g)
    ok = np.isfinite(g) & (res < 1e-8)
    if strict and not ok.all(): raise RuntimeError("Newton không hội tụ")
    return g if strict else (g, ok)

def _xi(spec, x, w, N, eta):
    z = x - 1j * eta; g = (1 / (z[:, None] - spec[None, :])).mean(1); t = z *
g - 1
    gam, ok = _continuation(t, N * w, w, strict=False)
    gam[~ok] = (t / (1 + N * (w**2).sum() * t))[~ok]       # dự phòng (thực
tê´: 0 lâ`n trên HOSE)
    _STATS["fail"] += int((~ok).sum()); _STATS["total"] += len(ok)
    return x * gam.imag / t.imag

def _xi_outliers(lam, idx, w, N):
    a = N * w; out = np.empty(len(idx))
    for k, i in enumerate(idx):
        g = np.sum(1 / (lam[i] - np.delete(lam, i))) / N     # Stieltjes bo'-
một trên trục thực
        t = np.array([lam[i] * g - 1 + 0j])
        try:
            gam = _continuation(t, a, w); _, R, dR = _F(gam, t, a, w)
            out[k] = lam[i] * np.real(1 / (R + gam * dR))[0]   # ξ = λ·γ'(t)
        except RuntimeError:
            out[k] = _xi(lam, lam[i:i+1], w, N, 1 / np.sqrt(N))[0]
    return out

def _null_spectrum(N, w, reps=8, seed=0):
    key = (N, len(w), round(float(w[-1]), 12))
    if key not in _NULL_CACHE:
        rng = np.random.default_rng(seed)
        ev = [np.linalg.eigvalsh((Y * w[:, None]).T @ Y) for Y in
(rng.standard_normal((len(w), N)) for _ in range(reps))]
        _NULL_CACHE[key] = np.sort(np.concatenate(ev))
    return _NULL_CACHE[key]

def ew_rie_corr(C, w, debias=True):
    N = len(C); lam, V = np.linalg.eigh(C); lam = np.maximum(lam, 1e-10); eta
= 1 / np.sqrt(N)
    null = _null_spectrum(N, w); s2 = np.median(lam) / np.median(null)
    D = lam[:, None] - lam[None, :]
    dens = (eta / (D**2 + eta**2)).sum(1) - 1 / eta            # mật độ lân
cận (bo' tự thân)
```

```
    out = np.where(dens < 0.1 / eta)[0]; bulk = np.setdiff1d(np.arange(N),
out)
    xi = np.empty(N)
    xi[bulk] = _xi(lam, lam[bulk], w, N, eta)
    if len(out): xi[out] = _xi_outliers(lam, out, w, N)
    if debias and len(bulk):
        xi_null = _xi(null, lam[bulk] / s2, w, N, eta)
        xi[bulk] *= np.maximum(1, 1 / np.maximum(xi_null, 1e-6))
    xi = np.maximum(xi, 1e-8); xi *= N / xi.sum()
    return renorm((V * xi) @ V.T)
```

In [6]:

```
# ---------- Danh sách bộ ước lượng ----------
def build_estimators(cfg):
    hl, hv = cfg["HL_EW"], cfg["HL_VOL"]
    E = {
        "Sample": est_sample,
        "Ledoit-Wolf": est_lw,
        "NL-LW": est_nllw,
        "MP-clip": est_mp,
        "RIE": est_rie,
        "VS-NL-LW": make_vs(lambda Z: renorm(nl_lw_S(np.cov(Z, rowvar=False),
len(Z) - 1)), hv),
        "VS-RIE": make_vs(lambda Z: rie_corr(np.corrcoef(Z, rowvar=False),
Z.shape[1] / len(Z)), hv),
        "EW-NL-LW": make_ew(lambda C, w: renorm(nl_lw_S(C, 1 /
(w**2).sum())), hl),
        "EW-RIE (q_eff)": make_ew(lambda C, w: rie_corr(C, len(C) *
(w**2).sum()), hl),
        "EW-RIE*": make_ew(ew_rie_corr, hl),
    }
    if cfg["QUICK"]:
        E = {k: E[k] for k in ["Sample", "NL-LW", "RIE", "EW-NL-LW", "EW-
RIE*"]}
    return E
```

▶ v3 · §1c. Bổ sung (ô mới — không sửa các ô §1 của thầy)

- `nl_lw_S` được định nghĩa lại để đúng khi **N > T_eff** (bản của thầy giữ làm `nl_lw_S_v2`). Khi N ≤ T_eff kết quả trùng hệt.
- **EW-WeSpeR** (Oriol 2024, arXiv:2410.14413): khôi phục phổ quần thể rồi co phi tuyến (kiểu QuEST, có trọng số). Cài lại bằng numpy từ các phương trình của bài.
- `build_estimators` được bọc để thêm "EW-WeSpeR"; ô §4 của thầy chạy nguyên văn.

In [7]:

```
import hashlib
nl_lw_S_v2 = nl_lw_S        # bản của thầy (chỉ đúng khi N ≤ T_eff), giữ để đối chứng

# ---------- v3: NL-LW cho cả trường hợp p > n (N > T_eff) ----------
def nl_lw_S(S, n):
    # Ledoit & Wolf (2020), co phi tuyến giải tích; S: hiệp phương sai, n: cỡ mẫu (hiệu dụng, có thể không nguyên).
    # p <= n: công thức gốc (giống hệt v2).  p > n: nhánh "p > n" trong mã gốc analytical_shrinkage.m của LW (2020):
    # chỉ [n] trị riêng lớn nhất được xem là "khác 0" (dùng cho mật độ nhân), các trị riêng còn lại nhận chung d̃₀
    # (d̃₀ dựa trên Hilbert transform tại 0). v2 dùng công thức p <= n với c = p/n > 1 ⇒ mẫu số (1 − c − πcλH)² sai, co sai các trị riêng nhỏ.
    p = len(S); lam, U = np.linalg.eigh(S); lam = np.maximum(lam, 1e-12)
    m = p if p <= n else max(int(np.floor(n)), 2)
    lz = lam[p - m:]; h = n ** (-1 / 3)
    L = np.tile(lz, (m, 1)).T; H = h * L.T; x = (L - L.T) / H
    ft = (3 / 4 / np.sqrt(5)) * np.mean(np.maximum(1 - x**2 / 5, 0) / H, axis=1)
    with np.errstate(divide="ignore", invalid="ignore"):
        Hf = (-3/10/np.pi) * x + (3/4/np.sqrt(5)/np.pi) * (1 - x**2/5) * np.log(np.abs((np.sqrt(5) - x) / (np.sqrt(5) + x)))
    e = np.isclose(np.abs(x), np.sqrt(5)); Hf[e] = (-3/10/np.pi) * x[e]
    Ht = np.mean(Hf / H, axis=1); c = p / n
    if p <= n:
        d = lz / ((np.pi*c*lz*ft)**2 + (1 - c - np.pi*c*lz*Ht)**2)
    else:
        Hf0 = (1/np.pi) * (3/10/h**2 + 3/4/np.sqrt(5)/h * (1 - 1/5/h**2) * np.log((1 + np.sqrt(5)*h) / (1 - np.sqrt(5)*h))) * np.mean(1 / lz)
        d0 = 1 / (np.pi * (p - n) / n * Hf0)
        d1 = lz / (np.pi**2 * lz**2 * (ft**2 + Ht**2))
        d = np.concatenate([np.full(p - m, d0), d1])
    return (U * d) @ U.T
```

In [8]:
```
# ---------- v3: EW-WeSpeR — đổi chứng kiểu QuEST cho hiệp phương sai có trọng số (Oriol 2024, arXiv:2410.14413) ----------
# Mô hình: E = Σ_t w_t x_t x_tᵀ, x_t = C^{1/2} y_t, Σw = 1. Với r(e) = Σ_t w_t / (1 + N w_t e) (= R_W(−e)):
#   e(z) = (1/N) Σ_i τ_i / (τ_i r(e(z)) − z),      m(z) = (1/N) Σ_i 1 / (τ_i r(e(z)) − z)     (Im z > 0)
# m là Stieltjes của phổ mẫu tiệm cận (mật độ ρ = Im m / π); e = (1/N) tr C·G(z). Cơ phi tuyến "oracle":
#   d(λ) = lim u_λᵀ C u_λ = Im e(λ + i0) / Im m(λ + i0)      (trọng số đều ⇒ λ / |1 − q − qλ m|², Ledoit–Péché).
# WeSpeR = (1) tìm phổ quần thể τ sao cho phân vị của ρ_τ khớp trị riêng mẫu (như QuEST, nhưng với trọng số),
#          (2) áp d(λ_i) với τ đã khớp. Khác EW-RIE*: EW-RIE* dùng trực tiếp Stieltjes thực nghiệm (làm trơn η = N^{-1/2}),
#          WeSpeR dùng Stieltjes của MÔ HÌNH đã khớp ⇒ không cần η, không cần xử lý ngoại lai/hiệu chỉnh N hữu hạn riêng.
# Ghi chú: cài lại bằng numpy từ các phương trình trên (bản gốc của tác giả viết bằng PyTorch + autodiff; phiên làm việc này
# không truy cập được arXiv). τ tham số hóa: log τ tuyến tính từng khúc theo hạng (K nút) + các ngoại lai tự do;
# tối ưu bằng least_squares (Levenberg–Marquardt kiểu trf), sai số tương đối log λ̂_i − log λ_i; khởi tạo ấm từ kỳ trước.
_WSP = {"nfev": [], "cost": [], "warm": {}}

def _wsp_bins(w, nb=40):
    # nén phân phối trọng số thành nb nhóm (r chỉ phụ thuộc phân phối của w): giá trị TB và khối lượng Σw mỗi nhóm
    ws = np.sort(w); g = np.array_split(ws, min(nb, len(ws)))
    return np.array([x.mean() for x in g]), np.array([x.sum() for x in g])

def _wsp_solve(z, tau, wv, wm, N, e0=None, iters=40):
    # Newton (có tìm kiếm lùi, giữ Im e > 0) cho F(e) = e − mean τ/(τ r(e) − z), vector theo z
    def F(e):
        den = 1 + N * wv[None, :] * e[:, None]
        r = (wm[None, :] / den).sum(1); dr = -(N * wm[None, :] * wv[None, :] / den**2).sum(1)
        q = tau[None, :] * r[:, None] - z[:, None]
        f = e - (tau[None, :] / q).mean(1)
        df = 1 + (tau[None, :]**2 * dr[:, None] / q**2).mean(1)
        return f, df, q
    e = (tau[None, :] / (tau[None, :] - z[:, None])).mean(1) if e0 is None else e0.copy()
    f, df, q = F(e)
    for _ in range(iters):
        st = f / df; a = np.ones(len(e))
        for _ in range(25):
            en = e - a * st; fn, dfn, qn = F(en)
            bad = (~np.isfinite(fn)) | (np.abs(fn) > np.abs(f)) | (en.imag <= 0)
            bad &= np.abs(f) > 1e-13
            if not bad.any(): break
            a[bad] *= 0.5
        keep = bad
        en[keep], fn[keep], dfn[keep], qn[keep] = e[keep], f[keep], df[keep], q[keep]
        e, f, df, q = en, fn, dfn, qn
        if np.max(np.abs(f)) < 1e-11: break
    return e, (1 / q).mean(1), np.abs(f)

def _wsp_cold(x, eta, tau, wv, wm, N, steps=14):
    # tiếp tục theo η: từ η lớn (e ≈ mean τ/(τ − z)) xuống η đích ⇒ đúng nhánh Stieltjes (Im e > 0)
    e = None
    for s in np.geomspace(10 * max(x.max(), tau.max()), eta.min(), steps):
        e, m, res = _wsp_solve(x + 1j * np.maximum(eta, s), tau, wv, wm, N, e)
    return _wsp_solve(x + 1j * eta, tau, wv, wm, N, e)

def _wsp_quantiles(x, m, n_nz, N):
    # phân vị của phần khối lượng dương của ρ = Im m/π trên lưới x (hình thang), khớp với n_nz trị riêng khác 0
```

```
    rho = np.maximum(m.imag, 0) / np.pi; cdf = np.concatenate([[0],
np.cumsum(0.5 * (rho[1:] + rho[:-1]) * np.diff(x))])
    cdf /= cdf[-1]; u = (np.arange(n_nz) + 0.5) / n_nz
    return np.interp(u, cdf, x)

def _wsp_fwd(x, eta, tau, wv, wm, N, e0=None):
    # giải e trên lưới (ấm nếu có e0, nếu không/không hội tụ thì tiếp
tục theo η) + các đạo hàm theo τ (đạo hàm ẩn)
    ok = False
    if e0 is not None:
        e, m, res = _wsp_solve(x + 1j * eta, tau, wv, wm, N, e0, iters=15);
ok = np.isfinite(e).all() and res.max() < 1e-9
    if not ok: e, m, res = _wsp_cold(x, eta, tau, wv, wm, N)
    return e, m, res

def _wsp_jac_m(x, eta, e, tau, wv, wm, N):
    # dm/dτ_j (G×N):  F(e,τ) = e − mean τ/q, q = τ r(e) − z
    z = x + 1j * eta; den = 1 + N * wv[None, :] * e[:, None]
    r = (wm[None, :] / den).sum(1); dr = -(N * wm[None, :] * wv[None, :] /
den**2).sum(1)
    q = tau[None, :] * r[:, None] - z[:, None]; q2 = q**2
    dF = 1 + (tau[None, :]**2 * dr[:, None] / q2).mean(1)
    de = -(z[:, None] / (N * q2)) / dF[:, None]
    A = -(tau[None, :] * dr[:, None] / q2).mean(1)
    return -(r[:, None] / (N * q2)) + A[:, None] * de

def ew_wesper_corr(C, w, K=8, max_nfev=25, G=350, return_info=False):
    from scipy.optimize import least_squares
    N = len(C); lam, V = np.linalg.eigh(C); T = len(w)
    n_nz = min(N, T - 1); lz = np.maximum(lam[N - n_nz:], 1e-10)          #
trị riêng khác 0 (tăng dần)
    wv, wm = _wsp_bins(w)
    eta0 = 1 / np.sqrt(N); D = lz[:, None] - lz[None, :]                    #
ngoại lai: như ew_rie_corr
    dens = (eta0 / (D**2 + eta0**2)).sum(1) - 1 / eta0
    n_out = int(min(np.sum(dens < 0.1 / eta0), 5)); nb = N - n_out
    knots = np.linspace(0, 1, K); rk = (np.arange(nb) + 0.5) / nb
    def tau_of(th):
        tb = np.exp(np.interp(rk, knots, np.cumsum(np.concatenate([[th[0]],
np.exp(th[1:K])]))))
        return np.concatenate([tb, np.exp(th[K:])]) if n_out else tb
    x = np.unique(np.concatenate([np.geomspace(lz[0] * .25, lz[-1] * 1.6, G),
                                  *[lz[-k] * np.linspace(.6, 1.4, 60) for k
in range(1, n_out + 1)]]))
    eta = 2e-3 * x; dx = np.diff(x); u = (np.arange(n_nz) + 0.5) / n_nz
    key = (N, T, K, n_out, round(float(w[-1]), 12))
    qb = np.maximum.accumulate(np.log(np.quantile(lz[:n_nz - n_out] if n_nz >
n_out else lz, knots)))
    th_cold = np.concatenate([[qb[0]], np.log(np.maximum(np.diff(qb), 1e-3)),
np.log(lz[n_nz - n_out:] * .9) if n_out else []])
    st = {}
    def evaluate(th):
        if st.get("th") is not None and np.array_equal(st["th"], th): return
        tau = tau_of(th); e, m, res = _wsp_fwd(x, eta, tau, wv, wm, N,
st.get("e"))
        rho = np.maximum(m.imag, 0) / np.pi; raw = np.concatenate([[0],
np.cumsum(0.5 * (rho[1:] + rho[:-1]) * dx)])
        tot = raw[-1]; cdf = raw / tot; qh = np.interp(u, cdf, x)
        st.update(th=th.copy(), tau=tau, e=e, m=m, rho=rho, raw=raw, tot=tot,
cdf=cdf, qh=qh, ok=np.isfinite(qh).all())
    def fun(th):
        evaluate(th); return np.log(st["qh"]) - np.log(lz) if st["ok"] else
np.full(n_nz, 10.0)
    def jac(th):
        evaluate(th)
        dm = _wsp_jac_m(x, eta, st["e"], st["tau"], wv, wm, N); dr = dm.imag
/ np.pi
        draw = np.vstack([np.zeros(N), np.cumsum(0.5 * (dr[1:] + dr[:-1]) *
dx[:, None], axis=0)])
        dcdf = (draw - st["cdf"][:, None] * draw[-1][None, :]) / st["tot"]
        k = np.clip(np.searchsorted(x, st["qh"]) - 1, 0, len(x) - 2); f =
(st["qh"] - x[k]) / dx[k]
        dC = dcdf[k] * (1 - f)[:, None] + dcdf[k + 1] * f[:, None]
```

```
        pdf = np.maximum((st["rho"][k] * (1 - f) + st["rho"][k + 1] * f) /
st["tot"], 1e-3 / (lz[-1]))
        Jt = -dC / pdf[:, None] / st["qh"][:, None]
# ∂ log λˆ / ∂τ
        h = 1e-6; t0 = tau_of(th)
        Tt = np.column_stack([(tau_of(th + h * np.eye(len(th))[j]) - t0) / h
for j in range(len(th))])
        return Jt @ Tt
    fit = lambda th0: (st.clear(), least_squares(fun, th0, jac=jac,
method="trf", max_nfev=max_nfev, xtol=1e-4, ftol=1e-4))[1]
    sol = fit(_WSP["warm"].get(key, th_cold))                # khởi tạo â´m
từ kỳ trước (cùng N, T, h) nê´u có
    if key in _WSP["warm"] and np.sqrt(np.mean(sol.fun**2)) > 0.05:  # â´m
hởng (cực tiểu địa phương) ⇒ khớp lại từ đầ̀u, giữ nghiệm tô´t hơn
        s2 = fit(th_cold); sol = s2 if np.sum(s2.fun**2) < np.sum(sol.fun**2)
else sol
    _WSP["warm"][key] = sol.x; _WSP["nfev"].append(sol.nfev);
_WSP["cost"].append(np.sqrt(np.mean(sol.fun**2)))
    tau = tau_of(sol.x)
    # co: d(λ_i) = Im e / Im m tại λ_i + iη (η nhở; ở khe/ngoài giá đỡ tỷ̉
sô´ → e'(λ)/m'(λ), vâ˜n xác định)
    ee, mm, _ = _wsp_cold(lz, 1e-4 * lz, tau, wv, wm, N); d = ee.imag /
mm.imag
    xi = np.empty(N); xi[N - n_nz:] = np.maximum(d, 1e-8)
    if n_nz < N:   # không gian null (N ≥ T): giữ vê´t (tr C = N) cho các
hướng null
        xi[:N - n_nz] = max((N - xi[N - n_nz:].sum()) / (N - n_nz), 1e-3 *
np.mean(xi[N - n_nz:]))
    xi *= N / xi.sum(); out = renorm((V * xi) @ V.T)
    return (out, dict(tau=tau, xi=xi, lam=lam, rmse=_WSP["cost"][-1],
nfev=sol.nfev, n_out=n_out)) if return_info else out


# Bọc an toàn + cache theo (C, w): cùng cửa sổ ở hai chê´ độ chỉ̉ ước
lượng 1 lần; lô˜i sô´ ⇒ dự phòng EW-RIE* (được đê´m)
_WSP_SAFE = {"fallback": 0, "total": 0}; _WSP_CACHE = {}
def ew_wesper_safe(C, w):
    key = (hashlib.md5(C.tobytes()).hexdigest(), len(w), round(float(w[-1]),
14))
    if key not in _WSP_CACHE:
        _WSP_SAFE["total"] += 1
        try:
            out = ew_wesper_corr(C, w); assert np.isfinite(out).all()
        except Exception:
            _WSP_SAFE["fallback"] += 1; out = ew_rie_corr(C, w)
        _WSP_CACHE[key] = out
    return _WSP_CACHE[key]

_build_estimators_thay = build_estimators           # danh sách cửa thầy
def build_estimators(cfg):
    E, out = _build_estimators_thay(cfg), {}
    for k, f in E.items():
        if k == "EW-RIE*": out["EW-WeSpeR"] = make_ew(ew_wesper_safe,
cfg["HL_EW"])
        out[k] = f
    return out
```

1b. Kiểm thử (chạy trước mọi thay đổi)

1. Trọng số đều ⇒ EW-RIE* trùng RIE cổ điển. 2) So với oracle trên dữ liệu tổng hợp đã biết C thật.

In [9]:
```
rng = np.random.default_rng(0); N_, T_ = 100, 250
X = rng.standard_normal((T_, N_)); Cs = np.corrcoef(X, rowvar=False); lam =
np.linalg.eigvalsh(Cs)
z = lam - 0.1j; g = (1 / (z[:, None] - lam[None, :])).mean(1); q = N_ / T_
err = np.max(np.abs(_xi(lam, lam, np.full(T_, 1 / T_), N_, 0.1) - lam /
np.abs(1 - q + q*z*g)**2) / lam)
print(f"[Test 1] trọng số đều: sai lệch tương đối max = {err:.1e}"); assert
err < 1e-10
```

```
[Test 1] trọng số đều: sai lệch tương đối max = 2.9e-15
```

In [10]:
```
def gmv(S):
    w = np.linalg.solve(S, np.ones(len(S))); return w / w.sum()

def factor_corr(N, rng):
    sec = rng.integers(0, 8, N); B = np.zeros((N, 9))
    B[:, 0] = rng.uniform(.4, .8, N); B[np.arange(N), 1 + sec] =
rng.uniform(.2, .5, N)
    return renorm(B @ B.T + np.diag(rng.uniform(.3, .6, N)))

rows = []
for hl in (30, 60):
    w = ew_weights(T_, hl); acc = {}
    for rep in range(6):
        Ct = factor_corr(N_, rng); X = rng.standard_normal((T_, N_)) @
np.linalg.cholesky(Ct).T
        C, _ = ew_corr(X, w); lam, V = np.linalg.eigh(C)
        Co = renorm((V * np.einsum("ij,jk,ki->i", V.T, Ct, V)) @ V.T)    #
oracle RIE
        wo = gmv(Ct); best = wo @ Ct @ wo
        for k, Ch in {"EW-Sample": C, "EW-RIE (q_eff)": rie_corr(C, N_ *
(w**2).sum()),
                      "EW-NL-LW": renorm(nl_lw_S(C, 1 / (w**2).sum())), "EW-
RIE*": ew_rie_corr(C, w),
                      "Oracle": Co}.items():
            wh = gmv(Ch); acc.setdefault(k, []).append(wh @ Ct @ wh / best)
    rows += [dict(hl=hl, method=k, risk_ratio=np.mean(v)) for k, v in
acc.items()]
t2 = pd.DataFrame(rows).pivot(index="method", columns="hl",
values="risk_ratio").round(3)
print("[Test 2] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân
tố:"); display(t2)
assert (t2.loc["EW-RIE*"] < t2.loc["EW-RIE (q_eff)"]).all()
```

```
[Test 2] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân tố:
```

hl	30	60
method		
EW-NL-LW	1.636	1.269
EW-RIE (q_eff)	1.782	1.629
EW-RIE*	1.290	1.210
EW-Sample	2.929	2.048
Oracle	1.278	1.201

▶ v3 · §1b (bổ sung) — Test 2b: N > T_eff và WeSpeR

Như Test 2 của thầy nhưng thêm half-life 15 (N = 100 > T_eff ≈ 43), bản NL-LW cũ/mới và EW-WeSpeR.

In [11]:
```
def gmv(S):
    w = np.linalg.solve(S, np.ones(len(S))); return w / w.sum()

def factor_corr(N, rng):
    sec = rng.integers(0, 8, N); B = np.zeros((N, 9))
    B[:, 0] = rng.uniform(.4, .8, N); B[np.arange(N), 1 + sec] =
rng.uniform(.2, .5, N)
    return renorm(B @ B.T + np.diag(rng.uniform(.3, .6, N)))

rows = []
for hl in (15, 30, 60):
    w = ew_weights(T_, hl); acc = {}
    for rep in range(6):
        Ct = factor_corr(N_, rng); X = rng.standard_normal((T_, N_)) @
np.linalg.cholesky(Ct).T
        C, _ = ew_corr(X, w); lam, V = np.linalg.eigh(C)
        Co = renorm((V * np.einsum("ij,jk,ki->i", V.T, Ct, V)) @ V.T)    #
oracle RIE
        wo = gmv(Ct); best = wo @ Ct @ wo
        for k, Ch in {"EW-Sample": C, "EW-RIE (q_eff)": rie_corr(C, N_ *
(w**2).sum()),
                      "EW-NL-LW (v2)": renorm(nl_lw_S_v2(C, 1 /
(w**2).sum())), "EW-NL-LW": renorm(nl_lw_S(C, 1 / (w**2).sum())),
                      "EW-WeSpeR": ew_wesper_corr(C, w), "EW-RIE*":
ew_rie_corr(C, w), "Oracle": Co}.items():
            wh = gmv(Ch); acc.setdefault(k, []).append(wh @ Ct @ wh / best)
    rows += [dict(hl=hl, method=k, risk_ratio=np.mean(v)) for k, v in
acc.items()]
t2b = pd.DataFrame(rows).pivot(index="method", columns="hl",
values="risk_ratio").round(3)
teff = {hl: 1 / (ew_weights(T_, hl)**2).sum() for hl in (15, 30, 60)}
print("[Test 2b] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân tố,
N=100, T=250; T_eff =",
      {h: round(v) for h, v in teff.items()}); display(t2b)
assert (t2b.loc["EW-RIE*", [30, 60]] < t2b.loc["EW-RIE (q_eff)", [30,
60]]).all()
assert (t2b.loc["EW-NL-LW"] <= t2b.loc["EW-NL-LW (v2)"] + 1e-9).all()
# bản sửa không kém v2
assert np.isclose(t2b.loc["EW-NL-LW", 60], t2b.loc["EW-NL-LW (v2)", 60])
# N < T_eff: trùng v2
assert (t2b.loc["EW-WeSpeR"] / t2b.loc["Oracle"] < 1.03).all()
# WeSpeR trong 3% của oracle
```

```
[Test 2b] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân tố, N=100, T=
250; T_eff = {15: 43, 30: 86, 60: 155}
```

hl	15	30	60
method			
EW-NL-LW	1.422	1.744	1.183
EW-NL-LW (v2)	12.441	1.849	1.183
EW-RIE (q_eff)	1.741	1.753	1.573
EW-RIE*	1.718	1.313	1.152
EW-Sample	5.598	2.797	1.881
EW-WeSpeR	1.386	1.317	1.153
Oracle	1.367	1.301	1.147

Nhận xét Test 2b (và Test 2 của thầy).

- **N > T_eff (h = 15, T_eff ≈ 43):** EW-NL-LW bản của thầy cho rủi ro gấp **12,4 lần** tối ưu; bản sửa còn 1,42. EW-WeSpeR 1,39 gần oracle (1,37). EW-RIE* 1,72 kém hơn WeSpeR và EW-NL-LW (bản sửa) ở vùng q_eff > 2, nhưng vẫn tốt hơn EW-Sample (5,6) và EW-RIE (q_eff) (1,74).
- **h = 30 (T_eff ≈ 86, q_eff = 1,16):** bản sửa tốt hơn bản cũ (1,744 so với 1,849); EW-RIE* 1,313 và EW-WeSpeR 1,317 sát oracle (1,301).
- **h = 60 (N < T_eff):** hai bản NL-LW **trùng nhau** (1,183) đúng thiết kế; EW-RIE* 1,152 ≈ WeSpeR 1,153 ≈ oracle 1,147.
- **Test 2 của thầy (ô ngay trên, giữ nguyên mã):** nó gọi `nl_lw_S` nên giờ dùng bản sửa. Ở h = 30 cột EW-NL-LW là 1,636 (bản gốc của thầy in 1,822) vì N > T_eff; các cột khác không đổi.

2. Dữ liệu

Ba lựa chọn qua `DATA_SOURCE`. Kết quả chuẩn: `P` (giá đóng cửa điều chỉnh, dòng = ngày, cột = mã) và `V` (giá trị giao dịch, cùng kích thước).

▶ v3 · T1.0 — lấy dữ liệu `data_hose/`

Có sẵn cạnh notebook thì dùng; không thì tải 5 file từ GitHub (repo public). Hàm nạp của thầy (ô kế tiếp, không sửa) đọc `prices.csv`, `values.csv` ở thư mục hiện hành nên ô này đổi thư mục, ô sau đổi lại.

In [12]:
```
import urllib.request
RAW_URL =
"https://raw.githubusercontent.com/chinguyenkhac/data_HOSE/main/data_hose"
HOSE_FILES = ["prices.csv", "values.csv", "volumes.csv",
"security_master.csv", "missing_tickers.csv"]
if DATA_DIR is None or not all(os.path.exists(f"{DATA_DIR}/{f}") for f in
HOSE_FILES):
    DATA_DIR = os.path.abspath(f"{WORKDIR}/data_hose"); os.makedirs(DATA_DIR,
exist_ok=True)
    for f in HOSE_FILES:
        if not os.path.exists(f"{DATA_DIR}/{f}"):
urllib.request.urlretrieve(f"{RAW_URL}/{f}", f"{DATA_DIR}/{f}")
    print("Đã tải từ GitHub →", DATA_DIR)
else:
    print("Dùng dữ liệu có sẵn:", DATA_DIR)
_CWD0 = os.getcwd(); os.chdir(DATA_DIR)      # để hàm nạp của thầy đọc
"prices.csv", "values.csv"
```

```
Dùng dữ liệu có sẵn: /home/user/data_HOSE/data_hose
```

In [13]:
```
def sh(cmd, cwd=None):
    r = subprocess.run(cmd, shell=True, cwd=cwd, capture_output=True,
text=True)
    if r.returncode: print(r.stderr[-500:])
    return r.returncode

def load_github(workdir):
    # Giá ngày 2006–03/2020
(github.com/88d52bdba0366127fffca9dfa93895/vnstock-data);
    # danh sách mã HOSE (github.com/vuthanhdatt/web-indicator, năm 2022 → có
survivorship bias)
    px_repo, ls_repo = f"{workdir}/vnstock-data", f"{workdir}/web-indicator"
    if not os.path.isdir(f"{px_repo}/tickers"):
        sh(f"git clone -q --depth 1 --filter=blob:none --sparse
https://github.com/88d52bdba0366127fffca9dfa93895/vnstock-data {px_repo}")
        sh("git sparse-checkout set tickers", cwd=px_repo)
    if not os.path.isdir(f"{ls_repo}/datas/hose"):
        sh(f"git clone -q --depth 1 --filter=blob:none --sparse
https://github.com/vuthanhdatt/web-indicator {ls_repo}")
        sh("git sparse-checkout set datas/hose", cwd=ls_repo)
    hose = {f[:-4].lower() for f in os.listdir(f"{ls_repo}/datas/hose")}
    px, val = {}, {}
    for f in sorted(os.listdir(f"{px_repo}/tickers")):
        t = f[:-4]
        if t not in hose: continue
        d = pd.read_csv(f"{px_repo}/tickers/{f}", parse_dates=
["date"]).drop_duplicates("date").set_index("date")
        d = d[d["close"] > 0]; px[t.upper()] = d["close"]; val[t.upper()] =
d["close"] * d["volume"]
    return pd.DataFrame(px).sort_index(), pd.DataFrame(val)

def load_csv(price_path, value_path=None):
    P = pd.read_csv(price_path, index_col=0, parse_dates=True).sort_index()
    V = pd.read_csv(value_path, index_col=0,
parse_dates=True).reindex(P.index) if value_path else P * 0 + 1
    return P, V

def load_vnstock(symbols, start="2006-01-01", end="2025-12-31"):
    # T1: API vnstock thay đổi theo phiên bản — KIỂM TRA lại trước khi
dùng (pip install -U vnstock).
    from vnstock import Vnstock
    px, val = {}, {}
    for s in symbols:
        try:
            d = Vnstock().stock(symbol=s,
source="VCI").quote.history(start=start, end=end, interval="1D")
            d = d.set_index(pd.to_datetime(d["time"]))
            px[s] = d["close"]; val[s] = d["close"] * d["volume"]   # LƯU Ý:
kiểm tra giá đã điều chỉnh chưa
        except Exception as e:
            print(s, "lỗi:", e)
        time.sleep(0.3)
    return pd.DataFrame(px).sort_index(), pd.DataFrame(val).sort_index()

if DATA_SOURCE == "github":
    P, V = load_github(WORKDIR)
elif DATA_SOURCE == "csv":
    P, V = load_csv("prices.csv", "values.csv")      # T1: thay đường dẫn
else:
    P, V = load_vnstock(["VNM", "FPT", "VCB"])         # T1: thay bằng danh
sách đầy đủ, gồm mã đã huỷ niêm yết

P = P[P.notna().sum(1) >= 20]; V = V.reindex(P.index)
print(f"{P.shape[1]} mã, {P.shape[0]} phiên: {P.index[0].date()} →
{P.index[-1].date()}")
```

```
493 mã, 2957 phiên: 2015-01-05 → 2026-11-10
```

▶ v3 · T1 — chỉ giữ quan sát trong khoảng niêm yết HOSE

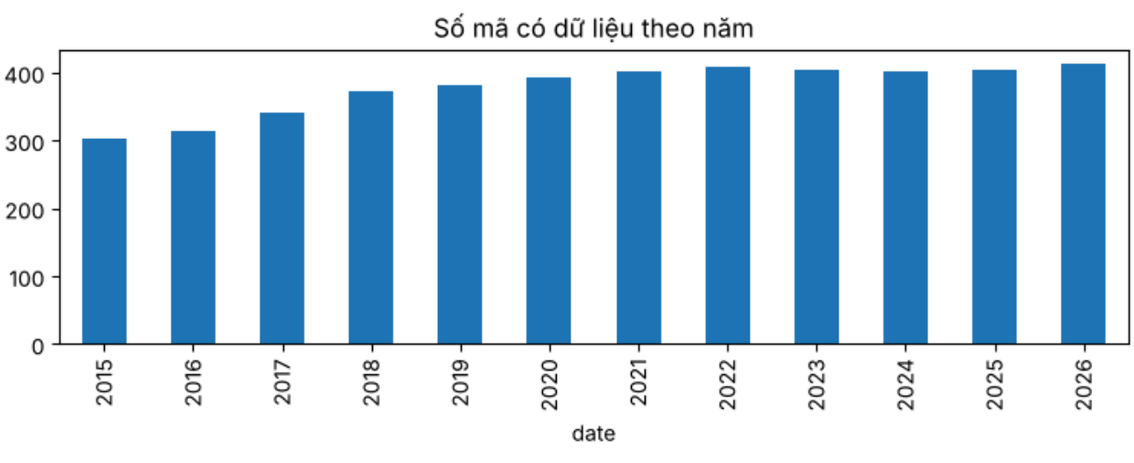
`security_master.csv`: mỗi dòng = một giai đoạn niêm yết trên HOSE (ngày niêm yết/hủy, lý do). Giá ngoài khoảng này bị loại; **mã hủy niêm yết được giữ** trong thời gian còn niêm yết.

In [14]:
```
os.chdir(_CWD0)
P_RAW = P.copy()
Q = pd.read_csv(f"{DATA_DIR}/volumes.csv", index_col=0,
parse_dates=True).reindex(index=P_RAW.index, columns=P_RAW.columns)
MASTER_RAW = pd.read_csv(f"{DATA_DIR}/security_master.csv", parse_dates=
["listing_date", "delisting_date"])
MISSING = pd.read_csv(f"{DATA_DIR}/missing_tickers.csv")
MASTER = MASTER_RAW.drop_duplicates(subset=["ticker", "listing_date",
"delisting_date"]).reset_index(drop=True)
INWIN = pd.DataFrame(False, index=P_RAW.index, columns=P_RAW.columns)      #
mặt nạ khoảng niêm yết (một mã có thể có nhiều giai đoạn)
for r in MASTER.itertuples():
    if r.ticker in INWIN:
        INWIN.loc[(INWIN.index >= r.listing_date) & ((INWIN.index <=
r.delisting_date) if pd.notna(r.delisting_date) else True), r.ticker] = True
OUTSIDE = P_RAW.notna() & ~INWIN
P = P_RAW.where(INWIN); V = V.reindex(index=P_RAW.index,
columns=P_RAW.columns).where(INWIN); Q = Q.where(INWIN)
P = P[P.notna().sum(1) >= 20]; V = V.reindex(P.index); Q = Q.reindex(P.index)
DELISTED = set(MASTER.loc[MASTER.delisting_date.notna(), "ticker"]) -
set(MASTER.loc[MASTER.delisting_date.isna(), "ticker"])
print(f"{P.shape[1]} mã có giá, {P.shape[0]} phiên: {P.index[0].date()} →
{P.index[-1].date()} | security_master: {MASTER.ticker.nunique()} mã,
{len(DELISTED)} mã đã rời HOSE | "
      f"thiếu giá: {len(MISSING)} mã | quan sát ngoài khoảng niêm yết (đã
loại): {int(OUTSIDE.sum().sum())}")
```

```
493 mã có giá, 2957 phiên: 2015-01-05 → 2026-11-10 | security_master: 512 mã, 1
04 mã đã rời HOSE | thiếu giá: 19 mã | quan sát ngoài khoảng niêm yết (đã loạ
i): 101
```

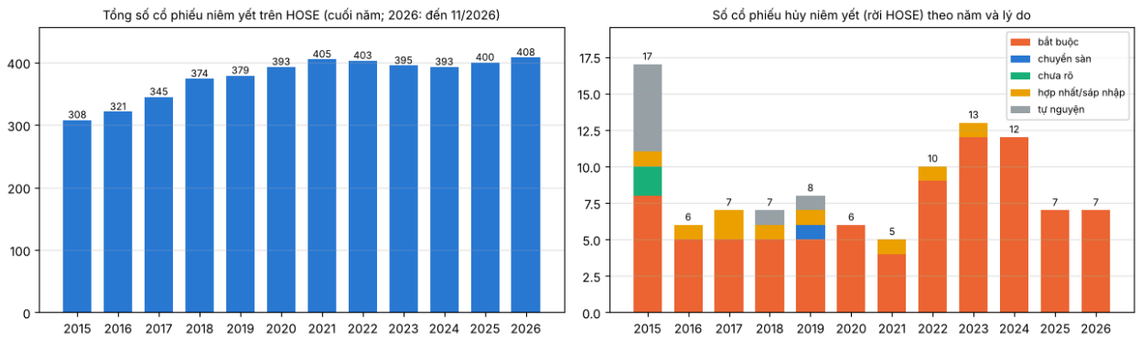
In [15]:
```
# Kiểm tra chất lượng dữ liệu
R_all = np.log(P).diff()
print("Tỷ lệ |r| > ngưỡng:", f"{(R_all.abs() >
CONFIG['MAX_ABS_RET']).sum().sum() / R_all.notna().sum().sum():.4%}")
print("Tỷ lệ phiên giá không đổi (r = 0):", f"{(R_all == 0).sum().sum() /
R_all.notna().sum().sum():.1%}")
P.notna().groupby(P.index.year).any().sum(axis=1).plot.bar(figsize=(9, 2.5),
title="Số mã có dữ liệu theo năm"); plt.show()
```

```
Tỷ lệ |r| > ngưỡng: 0.0005%
Tỷ lệ phiên giá không đổi (r = 0): 15.3%
```



T1 — số cổ phiếu niêm yết và số cổ phiếu hủy niêm yết theo năm (từ `security_master`).

In [16]:
```
RN = {"bat_buoc": "bắt buộc", "hop_nhat": "hợp nhất/sáp nhập", "tu_nguyen":
"tự nguyện", "chuyen_san": "chuyển sàn", "chua_ro": "chưa rõ"}
yrs = list(range(P.index[0].year, P.index[-1].year + 1)); ends = {y:
min(pd.Timestamp(f"{y}-12-31"), P.index[-1]) for y in yrs}
tot = pd.Series({y: sum(1 for r in MASTER.itertuples() if r.listing_date <=
ends[y] and (pd.isna(r.delisting_date) or r.delisting_date >= ends[y])) for y
in yrs})
dl = MASTER.dropna(subset=["delisting_date"]); dl = dl[dl.delisting_date >=
P.index[0]]
DY = dl.assign(year=dl.delisting_date.dt.year,
reason=dl.reason.map(RN).fillna("chưa rõ")).pivot_table(index="year",
columns="reason", values="ticker", aggfunc="count",
fill_value=0).reindex(yrs, fill_value=0)
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].bar(tot.index, tot.values, color="#2a78d6", width=.7)
for x_, v in tot.items(): ax[0].text(x_, v + 3, str(v), ha="center",
fontsize=8)
ax[0].set_xticks(list(tot.index)); ax[1].set_xticks(yrs)
ax[0].set_title("Tổng số cổ phiếu niêm yết trên HOSE (cuối năm; 2026: đến
11/2026)", fontsize=10); ax[0].set_ylim(0, tot.max() * 1.12);
ax[0].grid(axis="y", alpha=.3)
bot = np.zeros(len(DY))
for col, c in zip(DY.columns, ["#eb6834", "#2a78d6", "#1baf7a", "#eda100",
"#9aa0a6"]):
    ax[1].bar(DY.index, DY[col], bottom=bot, color=c, width=.7, label=col);
bot += DY[col].values
for x_, v in zip(DY.index, bot): ax[1].text(x_, v + .3, str(int(v)),
ha="center", fontsize=8)
ax[1].set_title("Số cổ phiếu hủy niêm yết (rời HOSE) theo năm và lý do",
fontsize=10); ax[1].legend(fontsize=8); ax[1].grid(axis="y", alpha=.3);
ax[1].set_ylim(0, bot.max() * 1.15)
plt.tight_layout(); plt.show()
hp = dl.ticker.isin(P.columns)
print(f"Tổng {len(dl)} mã rời HOSE trong {yrs[0]}–{yrs[-1]}: {int(hp.sum())}
mã có giá (đã đưa vào dữ liệu), {int((~hp).sum())} mã không lấy được giá từ
VCI | "
      f"niêm yết {tot.iloc[0]} mã (cuối {yrs[0]}) → {tot.iloc[-1]} mã (cuối
{yrs[-1]})")
R_q = np.log(P).diff(); nq = int(R_q.notna().sum().sum())
print(f"Chất lượng: {100 * (R_q == 0).sum().sum() / nq:.1f}% phiên giá không
đổi (r = 0) | {100 * (P.notna() & INWIN.reindex(P.index)).sum().sum() /
INWIN.reindex(P.index).sum().sum():.1f}% ô có giá trong khoảng niêm yết | "
      f"{int(((Q == 0) & P.notna()).sum().sum())} phiên khối lượng = 0 mà có
giá | {int((P <= 0).sum().sum())} giá ≤ 0")
```




```
Tổng 105 mã rời HOSE trong 2015–2026: 86 mã có giá (đã đưa vào dữ liệu), 19 mã
không lấy được giá từ VCI | niêm yết 308 mã (cuối 2015) → 408 mã (cuối 2026)
Chất lượng: 15.3% phiên giá không đổi (r = 0) | 94.1% ô có giá trong khoảng niê
m yết | 0 phiên khối lượng = 0 mà có giá | 0 giá ≤ 0
```

Nhận xét T1.

- **Tổng số cổ phiếu** niêm yết trên HOSE tăng từ 308 (cuối 2015) lên đỉnh 405 (2021), rồi đi ngang 393–408 đến 11/2026.
- **105 mã rời HOSE** (5–17 mã/năm, không năm nào bằng 0): cao nhất 2015 (17), rồi 2023–2024 (13 và 12). Đa số hủy bắt buộc (85/105), còn lại hợp nhất (9), tự nguyện (8), chưa rõ (2), chuyển sàn (1). Phần lớn chuyển xuống UPCoM.
- **86/105 mã hủy có giá** nên đã nằm trong dữ liệu (không survivorship bias); **19 mã** VCI không còn giá (chủ yếu hủy 2015–2018, mã nhỏ). Vì vậy survivorship bias *chưa* loại hoàn toàn.
- **Chất lượng:** không giá ≤ 0, không phiên khối lượng = 0 mà vẫn có giá; 94,1% ô có giá trong khoảng niêm yết (phần còn lại là đình chỉ/không khớp, không nội suy). **101 quan sát** ngoài khoảng niêm yết (toàn mã SAP) đã loại. Chỉ 5 phiên $|r| > 15\%$ (đặt = 0).
- **15,3% phiên đứng giá (r = 0):** đặc trưng của mã kém thanh khoản, không phải lỗi dữ liệu, nhưng làm GMV đánh giá thấp rủi ro ⇒ lọc `ZERO_MAX = 20%` (T2b).
- **Mã hủy niêm yết:** lợi suất 0 sau ngày cuối; ở bản đầy đủ (`EW_RIE_HOSE_v3.ipynb`) chỉ 2 vị thế trong 118 kỳ × 100 mã bị hủy ngay trong kỳ nắm giữ, và bỏ mã đã hủy làm Vol thấp đi 0,26–0,39 pp, lợi suất cao lên 1,1–2,6 pp/năm.

⇒ Dữ liệu đủ dùng cho §4; hai hạn chế cần nêu trong bài: 19 mã hủy thiếu giá và giả định lợi suất 0 sau khi hủy.

3. Danh mục & backtest

Vũ trụ động: mỗi kỳ chọn N mã có giá trị GD trung vị cao nhất trong cửa sổ (≥95% phiên có dữ liệu). Mã ngừng GD ngoài mẫu → lợi suất 0.

In [17]:

```
def gmv_unconstrained(S):
    w = np.linalg.solve(S, np.ones(len(S))); return w / w.sum()

def gmv_long_only(S, iters=3000):
    # FISTA + chiếu lên đơn hình (khớp SLSQP, nhanh hơn)
    n = len(S)
    def proj(v):
        u = np.sort(v)[::-1]; css = np.cumsum(u) - 1
        k = np.nonzero(u - css / np.arange(1, n + 1) > 0)[0][-1]; return
np.maximum(v - css[k] / (k + 1), 0)
    L = 2 * np.linalg.eigvalsh(S)[-1]; w = y = np.full(n, 1 / n); t = 1.0
    for _ in range(iters):
        wn = proj(y - 2 * S @ y / L); tn = (1 + np.sqrt(1 + 4*t*t)) / 2
        y = wn + (t - 1) / tn * (wn - w)
        if np.abs(wn - w).max() < 1e-9: return wn
        w, t = wn, tn
    return w

PERIODS = [("2008-09 Khủng hoảng TC", "2008-01-01", "2009-12-31"), ("2010-12
Lạm phát/nợ xấu", "2010-01-01", "2012-12-31"),
           ("2013-17 Ổn định", "2013-01-01", "2017-12-31"), ("2018-19 Điều
chỉnh", "2018-01-01", "2019-12-31"),
           ("2020Q1 COVID", "2020-01-01", "2020-03-31"), ("2020-21 Bùng nổ",
"2020-04-01", "2021-12-31"),
           ("2022 Sụt giảm", "2022-01-01", "2022-12-31"), ("2023-25", "2023-
01-01", "2025-12-31")]

def run_backtest(P, V, estimators, cfg, long_only):
    N, W, REB = cfg["N"], cfg["WINDOW"], cfg["REB"]
    R = np.log(P).diff(); R = np.expm1(R.mask(R.abs() > cfg["MAX_ABS_RET"],
0.0))
    dates = R.index; names = list(estimators)
    rets = {k: [] for k in names}; idx = []; reb_dates = []
    Wp = {k: pd.Series(dtype=float) for k in names}
    st = {k: dict(turn=[], neff=[], wmax=[], short=[]) for k in names}
    for s in range(W + 1, len(dates) - 1, REB):
        win = R.iloc[s - W:s]; ok = (win.notna().mean() >= .95) & P.iloc[s -
1].notna()
        if ok.sum() < N: continue
        tick = V.iloc[s - W:s].loc[:,
ok].median().sort_values(ascending=False).index[:N]
        X = win[tick].fillna(0).values; Y = R.iloc[s:s + REB]
[tick].fillna(0).values
        idx.extend(dates[s:s + REB]); reb_dates.append(dates[s])
        for k in names:
            S = estimators[k](X)
            w = gmv_long_only(S) if long_only else gmv_unconstrained(S)
            ws = pd.Series(w, index=tick)
            if len(Wp[k]):
                u = ws.index.union(Wp[k].index)
                st[k]["turn"].append((ws.reindex(u).fillna(0) -
Wp[k].reindex(u).fillna(0)).abs().sum())
            Wp[k] = ws; rets[k].extend(Y @ w)
            st[k]["neff"].append(1 / (w**2).sum()); st[k]
["wmax"].append(w.max()); st[k]["short"].append(np.clip(-w, 0, None).sum())
    out = {}
    for k in names:
        r = pd.Series(rets[k], index=idx); rn = r.copy()
        rn.loc[reb_dates[1:]] -= cfg["COST"] * np.array(st[k]["turn"])
        out[k] = dict(r=r, r_net=rn, m={
            "Vol %": r.std() * np.sqrt(252) * 100, "LN/năm %": r.mean() * 252
* 100,
            "Sharpe": r.mean() / r.std() * np.sqrt(252), "Sharpe sau phí":
rn.mean() / rn.std() * np.sqrt(252),
            "Turnover/kỳ": np.mean(st[k]["turn"]), "Số mã hiệu dụng":
np.mean(st[k]["neff"]),
            "Tỷ trọng max %": np.mean(st[k]["wmax"]) * 100, "Short %":
np.mean(st[k]["short"]) * 100})
    return out

def block_bootstrap_ci(ra, rb, block=21, B=2000, seed=0):
    # CI 95% cho Vol(a) − Vol(b), %/năm. (T3: bổ sung kiểm định Ledoit–Wolf
```

```
2011)
    rng = np.random.default_rng(seed); n = len(ra); nb = n // block; d =
np.empty(B)
    for i in range(B):
        ix = (rng.integers(0, n - block, nb)[:, None] +
np.arange(block)).ravel()
        d[i] = (ra[ix].std() - rb[ix].std()) * np.sqrt(252) * 100
    return np.percentile(d, [2.5, 97.5])

def summarize(out, pairs):
    tab = pd.DataFrame({k: o["m"] for k, o in out.items()}).T
    per = pd.DataFrame({k: {p: o["r"][s:e].std() * np.sqrt(252) * 100 for p,
s, e in PERIODS if len(o["r"][s:e]) > 20}
                        for k, o in out.items()}).T
    ci = pd.DataFrame({f"{a} vs {b}": block_bootstrap_ci(out[a]["r"].values,
out[b]["r"].values)
                       for a, b in pairs if a in out and b in out}, index=
["CI 2.5%", "CI 97.5%"]).T
    ci["Có ý nghĩa"] = (ci["CI 97.5%"] < 0) | (ci["CI 2.5%"] > 0)
    return tab, per, ci
```

▶ v3 · §3b. Lọc mã đứng giá (`ZERO_MAX`) và giai đoạn thị trường — không sửa `run_backtest` của thầy

`run_backtest` bên dưới được **sinh từ chính mã nguồn của thầy** (`inspect.getsource`), chỉ chèn một dòng ngay trước `if ok.sum() < N: continue` : `ok &= (win == 0).mean() <= cfg.get("ZERO_MAX", 1.0)` (chỉ dùng cửa sổ quá khứ; kỳ không đủ N mã bị bỏ, không hạ N). Với `ZERO_MAX = 1.0` kết quả **trùng hệt** bản của thầy (assert). `PERIODS` được cập nhật theo mẫu 2016–2026 (dữ liệu từ 2015).

In [18]:
```
import inspect
ULOG = []
_src = inspect.getsource(run_backtest)
_chk = "        if ok.sum() < N: continue\n"
_sel = "        tick = V.iloc[s - W:s].loc[:,
ok].median().sort_values(ascending=False).index[:N]\n"
assert _src.count(_chk) == 1 and _src.count(_sel) == 1, "mã nguồn
run_backtest của thầy đã đổi"
exec(_src.replace("def run_backtest(", "def run_backtest_x(")
         .replace(_chk, '        ok &= (win == 0).mean() <=
cfg.get("ZERO_MAX", 1.0)          # v3: lọc mã đứng giá (cả sổ quá khứ)\n' +
_chk)
         .replace(_sel, _sel + "        ULOG.append(dict(date=dates[s],
n_eligible=int(ok.sum()), tickers=list(tick)))\n"))
run_backtest_thay = run_backtest                          # bản gốc của
thầy (đổi chiều)
def run_backtest(*a, **k):                                # các ô của thầy
gọi run_backtest → dùng bản có lọc
    ULOG.clear(); return run_backtest_x(*a, **k)

PERIODS = [("2016-17 Ổn định", "2016-01-01", "2017-12-31"), ("2018-19 Điều
chỉnh", "2018-01-01", "2019-12-31"),
           ("2020Q1 COVID", "2020-01-01", "2020-03-31"), ("2020-21 Bùng nổ",
"2020-04-01", "2021-12-31"),
           ("2022 Sụt giảm", "2022-01-01", "2022-12-31"), ("2023-24 Phục
hồi", "2023-01-01", "2024-12-31"),
           ("2025-26", "2025-01-01", "2026-12-31")]

_p, _v = P.iloc[-900:], V.iloc[-900:]                     # kiểm tra: ZERO_MAX
= 1.0 ⇒ trùng hệt bản của thầy
_a = run_backtest_thay(_p, _v, {"NL-LW": est_nllw}, CONFIG, True)["NL-LW"]
["r"]
_b = run_backtest(_p, _v, {"NL-LW": est_nllw}, dict(CONFIG, ZERO_MAX=1.0),
True)["NL-LW"]["r"]
assert _a.index.equals(_b.index) and np.array_equal(_a.values, _b.values)
print(f"run_backtest (có lọc) ≡ run_backtest của thầy khi ZERO_MAX = 1.0 ✔ |
cấu hình chuẩn: ZERO_MAX = {CONFIG['ZERO_MAX']:.0%}")
```

```
run_backtest (có lọc) ≡ run_backtest của thầy khi ZERO_MAX = 1.0 ✔ | cấu hình c
huẩn: ZERO_MAX = 20%
```

4. Chạy chính

Thời gian: chế độ đầy đủ trên Colab khoảng 15–30 phút (10 bộ ước lượng × 2 chế độ). Đặt `QUICK=True` để chạy thử nhanh.

In [19]:
```
ESTIMATORS = build_estimators(CONFIG)
PAIRS = [("EW-RIE*", "EW-RIE (q_eff)"), ("EW-RIE*", "EW-NL-LW"), ("EW-RIE*",
"NL-LW"), ("EW-RIE*", "RIE"),
         ("NL-LW", "Ledoit-Wolf"), ("VS-NL-LW", "NL-LW")]
RESULTS = {}
for long_only in (False, True):
    mode = "long_only" if long_only else "short_ok"; t0 = time.time()
    out = run_backtest(P, V, ESTIMATORS, CONFIG, long_only)
    RESULTS[mode] = dict(out=out, **dict(zip(["table", "periods", "ci"],
summarize(out, PAIRS))))
    print(f"\n===== {mode} | {time.time() - t0:.0f}s =====")
    display(RESULTS[mode]["table"].round(2))
    display(RESULTS[mode]["periods"].round(2))
    display(RESULTS[mode]["ci"].round(3))
print("Newton thất bại:", _STATS)
```

```
===== short_ok | 243s =====
```

	Vol %	LN/năm %	Sharpe	Sharpe sau phí	Turnover/kỳ	Số mã hiệu dụng	Tỷ trọng max %	Short %
Sample	14.58	2.52	0.17	-0.20	1.81	4.60	24.22	122.25
Ledoit-Wolf	13.63	2.55	0.19	-0.08	1.20	8.41	17.87	86.54
NL-LW	13.10	1.76	0.13	-0.05	0.81	11.52	13.98	60.78
MP-clip	13.48	1.92	0.14	-0.05	0.86	7.46	22.86	64.80
RIE	13.30	4.69	0.35	0.23	0.55	12.88	17.25	30.58
VS-NL-LW	12.49	3.15	0.25	-0.08	1.41	7.65	25.73	54.28
VS-RIE	12.63	4.15	0.33	0.09	0.99	11.22	21.34	29.08
EW-NL-LW	12.72	1.30	0.10	-0.13	1.00	8.04	22.77	58.70
EW-RIE (q_eff)	14.30	4.75	0.33	0.23	0.51	20.25	14.14	12.53
EW-WeSpeR	12.57	1.73	0.14	-0.08	0.92	8.67	21.82	53.59
EW-RIE*	12.56	1.01	0.08	-0.14	0.94	8.53	22.16	54.96

	2016-17 Ổn định	2018-19 Điều chỉnh	2020Q1 COVID	2020-21 Bùng nổ	2022 Sụt giảm	2023-24 Phục hồi	2025-26
Sample	11.11	11.85	16.85	15.15	18.09	12.39	17.27
Ledoit-Wolf	9.54	11.24	20.32	13.77	17.15	11.50	15.73
NL-LW	9.38	10.87	20.48	12.81	16.95	10.66	15.20
MP-clip	9.31	11.02	16.80	13.16	17.57	10.88	16.68
RIE	9.04	10.60	17.79	13.15	17.89	11.13	15.69
VS-NL-LW	8.33	10.11	14.67	10.65	17.49	9.91	16.09
VS-RIE	8.15	10.21	15.21	10.90	18.17	10.41	15.64
EW-NL-LW	8.78	10.42	16.65	11.60	17.25	9.97	16.03
EW-RIE (q_eff)	8.49	10.79	18.46	13.55	20.68	12.25	17.12
EW-WeSpeR	8.50	10.08	16.47	11.52	17.13	10.03	15.83
EW-RIE*	8.59	10.14	16.45	11.48	17.12	9.90	15.81

	CI 2.5%	CI 97.5%	Có ý nghĩa
EW-RIE* vs EW-RIE (q_eff)	-2.305	-1.121	True
EW-RIE* vs EW-NL-LW	-0.237	-0.094	True
EW-RIE* vs NL-LW	-0.990	-0.058	True
EW-RIE* vs RIE	-1.196	-0.208	True
NL-LW vs Ledoit-Wolf	-0.741	-0.319	True
VS-NL-LW vs NL-LW	-1.277	0.061	False

```
===== long_only | 90s =====
```

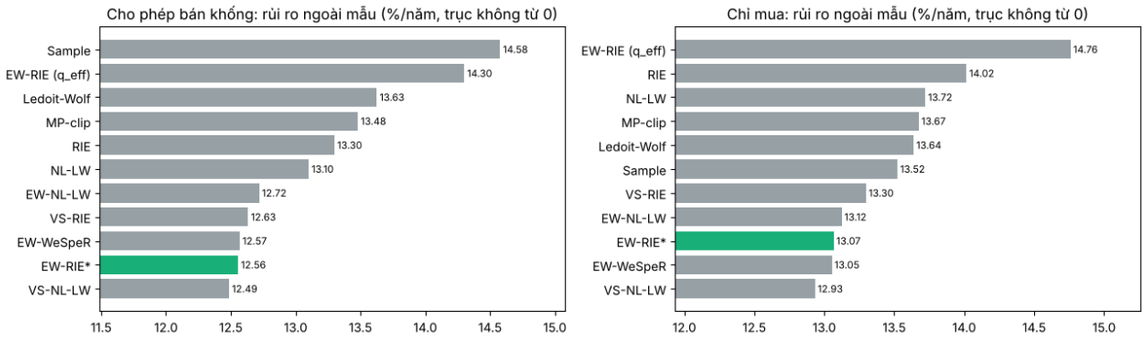
	Vol %	LN/năm %	Sharpe	Sharpe sau phí	Turnover/kỳ	Số mã hiệu dụng	Tỷ trọng max %	Short %
Sample	13.52	1.65	0.12	0.03	0.41	9.24	25.08	0.0
Ledoit-Wolf	13.64	2.36	0.17	0.09	0.38	12.63	20.06	0.0
NL-LW	13.72	2.08	0.15	0.08	0.34	14.40	16.80	0.0
MP-clip	13.67	3.36	0.25	0.16	0.41	9.09	25.93	0.0
RIE	14.02	4.44	0.32	0.24	0.35	15.32	17.52	0.0
VS-NL-LW	12.93	4.24	0.33	0.16	0.74	8.06	30.73	0.0
VS-RIE	13.30	4.73	0.36	0.20	0.68	12.83	21.99	0.0
EW-NL-LW	13.12	1.20	0.09	-0.02	0.49	9.17	26.22	0.0
EW-RIE (q_eff)	14.76	4.99	0.34	0.26	0.41	22.24	14.07	0.0
EW-WeSpeR	13.05	1.40	0.11	-0.00	0.47	9.39	25.74	0.0
EW-RIE*	13.07	1.31	0.10	-0.01	0.48	9.46	25.65	0.0

	2016-17 Ổn định	2018-19 Điều chỉnh	2020Q1 COVID	2020-21 Bùng nổ	2022 Sụt giảm	2023-24 Phục hồi	2025-26
Sample	9.61	12.06	17.02	13.21	17.58	11.41	15.53
Ledoit-Wolf	9.49	12.09	20.58	13.13	17.67	11.36	15.43
NL-LW	9.52	12.10	20.67	13.08	18.11	11.42	15.50
MP-clip	9.58	11.87	16.69	13.26	18.16	11.45	15.99
RIE	9.29	11.62	18.04	13.71	19.31	11.90	16.15
VS-NL-LW	8.66	11.27	14.13	10.76	18.78	10.71	15.77
VS-RIE	8.48	11.16	15.12	11.26	19.67	11.26	15.96
EW-NL-LW	8.84	11.45	16.14	11.79	17.77	11.08	15.77
EW-RIE (q_eff)	8.67	11.41	18.67	13.95	21.54	12.84	17.34
EW-WeSpeR	8.77	11.30	16.13	11.66	17.77	11.10	15.66
EW-RIE*	8.81	11.33	16.11	11.73	17.75	11.02	15.71

	CI 2.5%	CI 97.5%	Có ý nghĩa
EW-RIE* vs EW-RIE (q_eff)	-2.162	-1.220	True
EW-RIE* vs EW-NL-LW	-0.091	-0.025	True
EW-RIE* vs NL-LW	-1.030	-0.281	True
EW-RIE* vs RIE	-1.311	-0.564	True
NL-LW vs Ledoit-Wolf	-0.048	0.219	False
VS-NL-LW vs NL-LW	-1.481	-0.119	True

```
Newton thất bại: {'fail': 0, 'total': 52404}
```

In [20]:
```
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
for a, mode, ttl in [(ax[0], "short_ok", "Cho phép bán khống"), (ax[1],
"long_only", "Chỉ mua")]:
    v = RESULTS[mode]["table"]["Vol %"].sort_values()
    a.barh(v.index, v.values, color=["#1baf7a" if k == "EW-RIE*" else
"#9aa0a6" for k in v.index])
    a.set_xlim(v.min() - 1, v.max() + 0.5); a.set_title(f"{ttl}: rủi ro ngoài
mẫu (%/năm, trục không từ 0)")
    for i, x in enumerate(v.values): a.text(x, i, f" {x:.2f}", va="center",
fontsize=8)
plt.tight_layout(); plt.show()
```



Nhận xét §4 – kết quả chính (12/2016–11/2026, 118 kỳ, ZERO_MAX 20%).

- **Bán khống:**
  - Thấp nhất là **VS-NL-LW 12,49%**, rồi đến nhóm EW: **EW-RIE* 12,56%**, **EW-WeSpeR 12,57%**, VS-RIE 12,63%, EW-NL-LW 12,72%.
  - Các đối chứng không trọng số kém rõ: NL-LW 13,10%, RIE 13,30%, Ledoit-Wolf 13,63%, Sample 14,58%. EW-RIE (q_eff) 14,30% là bản EW tệ nhất.
- **Chỉ mua:**
  - **VS-NL-LW 12,93%** thấp nhất, rồi EW-WeSpeR 13,05%, EW-RIE* 13,07%, EW-NL-LW 13,12%.
  - NL-LW 13,72%; RIE 14,02% còn cao hơn ma trận mẫu (13,52%).
- **Ý nghĩa thống kê** (block bootstrap b = 21, cả hai chế độ):
  - EW-RIE* thấp hơn có ý nghĩa so với EW-RIE (q_eff), EW-NL-LW, NL-LW và RIE.
  - **EW-RIE* và EW-WeSpeR không khác nhau** (CI chứa 0, chênh ±0,01 pp). Đây là điều lý thuyết dự đoán: cả hai ước lượng cùng một đại lượng oracle $u_i^T C u_i$ cho hiệp phương sai có trọng số, chỉ khác cách tính (trực tiếp/RIE vs khôi phục phổ/QuEST).
- **So với v2** (không lọc đứng giá): mức Vol cao hơn ≈ 1,4 pp (mẫu thời gian cũng khác) vì universe đã loại các mã "giả ít rủi ro" (T2b). **Thứ hạng thay đổi**: v2 có EW-RIE* gần như dẫn đầu, v3 thì VS-NL-LW dẫn ở cả hai chế độ (VS vs NL-LW có ý nghĩa ở chỉ mua, chưa ở bán khống).
- **Kinh tế:** GMV có lợi suất thấp (1–5%/năm) và Sharpe sau phí ≈ 0. Mục tiêu của phương pháp là rủi ro, không phải lợi suất.

In [21]:
```
# Lưu kết quả (tải về hoặc chép vào Drive)
with pd.ExcelWriter(f"
{WORKDIR}/results_N{CONFIG['N']}_T{CONFIG['WINDOW']}_h{CONFIG['HL_EW']}.xlsx"
) as xw:
    for mode, d in RESULTS.items():
        d["table"].round(4).to_excel(xw, sheet_name=f"{mode}_tong_hop")
        d["periods"].round(3).to_excel(xw, sheet_name=f"{mode}_giai_doan")
        d["ci"].round(4).to_excel(xw, sheet_name=f"{mode}_CI")
    pd.Series(CONFIG).to_frame("giá trị").to_excel(xw, sheet_name="CONFIG")
print("Đã lưu vào", WORKDIR)
```

```
Đã lưu vào ./rmt_work
```

5. Độ nhạy half-life (T6 mở rộng thêm N, T)

In [22]:
```
RUN_SENS = False   # đặt True để chạy (~10–20 phút)
if RUN_SENS:
    rows = []
    for hl in (30, 60, 120):
        est = {"EW-NL-LW": make_ew(lambda C, w: renorm(nl_lw_S(C, 1 /
(w**2).sum())), hl), "EW-RIE*": make_ew(ew_rie_corr, hl)}
        for lo in (False, True):
            o = run_backtest(P, V, est, CONFIG, lo)
            l, u = block_bootstrap_ci(o["EW-RIE*"]["r"].values, o["EW-NL-LW"]
["r"].values)
            rows.append(dict(hl=hl, mode="long_only" if lo else "short_ok",
vol_EWNLLW=o["EW-NL-LW"]["m"]["Vol %"],
                             vol_EWRIE=o["EW-RIE*"]["m"]["Vol %"], CI_lo=l,
CI_hi=u))
    display(pd.DataFrame(rows).round(3))
```

6. Việc của nhóm (khung sẵn – điền tiếp)

In [23]:

```
import hashlib, json, platform, importlib.util
from IPython.display import Markdown
# ---- 6.0 Tiện ích chung cho T1c–T6 (tái dùng run_backtest, summarize,
block_bootstrap_ci) ----
from scipy.stats import norm
if importlib.util.find_spec("arch") is None:      # Colab chưa có sẵn `arch`
(dùng cho DCC-GARCH ở T3)
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "arch"],
check=True)
SEED = 0; np.random.seed(SEED)
RESULTS_DIR, FIGS_DIR, CACHE = "./results_gon", "./figs_gon",
"./results_gon/cache"
for d in (RESULTS_DIR, FIGS_DIR, CACHE): os.makedirs(d, exist_ok=True)
RECOMPUTE = False                        # True: chạy lại mọi backtest; False:
dùng cache results/cache/*.pkl nếu có
CORE, MODES = ["NL-LW", "EW-NL-LW", "EW-RIE*"], {"long_only": True,
"short_ok": False}
VN = {"long_only": "Chỉ mua", "short_ok": "Bán khống"}
COL = {"NL-LW": "#2a78d6", "EW-NL-LW": "#eb6834", "EW-RIE*": "#1baf7a", "EW-
WeSpeR": "#8e44ad", "EW-NL-LW (v2)": "#eda100"}

TIMES = {}
def check_cov(S, name=""):                        # đối xứng, PSD, không NaN
    ev = np.linalg.eigvalsh((S + S.T) / 2)
    assert np.isfinite(S).all() and np.allclose(S, S.T, rtol=0, atol=1e-10 *
np.abs(S).max()) and ev[0] > -1e-10 * ev[-1], name
    return ev
def timed(name, f):
    def g(X):
        t = time.perf_counter(); S = f(X); TIMES.setdefault(name,
[]).append(time.perf_counter() - t); check_cov(S, name); return S
    return g

def run(est, cfg=None, long_only=True, data=None, tag=""):
    # backtest có cache: khóa = tên bộ ước lượng + cfg + chế độ (+ tag dữ
liệu). Trả về out, log universe, thời gian, số lần Newton thất bại.
    cfg = cfg or CONFIG; Pd, Vd = data if data is not None else (P, V)
    f = f"{CACHE}/{hashlib.md5(repr((sorted(est), sorted(cfg.items()),
long_only) + ((tag,) if tag else ())).encode()).hexdigest()[:12]}.pkl"
    if os.path.exists(f) and not RECOMPUTE: d = pd.read_pickle(f)
    else:
        TIMES.clear(); ULOG.clear(); f0, n0 = _STATS["fail"], _STATS["total"]
        out = run_backtest(Pd, Vd, {k: timed(k, g) for k, g in est.items()},
cfg, long_only)
        d = dict(out=out, ulog=list(ULOG), sec={k: np.mean(v) for k, v in
TIMES.items()},
                 newton=(_STATS["fail"] - f0, _STATS["total"] - n0));
pd.to_pickle(d, f)
    idx = [o["r"].index for o in d["out"].values()]
    assert all(i.equals(idx[0]) for i in idx), "common-sample vi phạm"      #
cùng ngày OOS cho mọi phương pháp
    return d

net = lambda o, c: o["r"] - (c / CONFIG["COST"]) * (o["r"] - o["r_net"])   #
r_net ở mức phí c (phí không đổi tỷ trọng)
ann = lambda r: (r.std() * np.sqrt(252) * 100, r.mean() / r.std() *
np.sqrt(252))
def metrics(o, cost=None, dates=None):
    r, rn = o["r"], net(o, cost or CONFIG["COST"])
    if dates is not None: r, rn = r.loc[dates], rn.loc[dates]
    return {"Vol %": ann(r)[0], "Sharpe": ann(r)[1], "Sharpe sau phí":
ann(rn)[1], "LN/năm %": r.mean() * 252 * 100,
            "Turnover/kỳ": o["m"]["Turnover/kỳ"], "Số mã hiệu dụng": o["m"]
["Số mã hiệu dụng"], "Số ngày OOS": len(r)}

f2 = lambda x, d=2: f"{x:.{d}f}".replace(".", ",")
pv = lambda p: "< 0,001" if p < 1e-3 else f2(p, 3)
md = lambda s: display(Markdown(s))
def show(df, d=3): display(df.style.format(precision=d, decimal=",",
thousands=" ", na_rep="–"))
def save(df, name):
    assert np.isfinite(df.select_dtypes("number").values).all(), f"{name}:
```

```
NaN/inf"; df.to_csv(f"{RESULTS_DIR}/{name}.csv")
def savefig(fig, name): fig.savefig(f"{FIGS_DIR}/{name}.png", dpi=150,
bbox_inches="tight"); plt.show()

R_BT = np.log(P).diff(); R_BT = np.expm1(R_BT.mask(R_BT.abs() >
CONFIG["MAX_ABS_RET"], 0.0))   # đúng như trong run_backtest
POS = {d: i for i, d in enumerate(R_BT.index)}
BASE = run({k: ESTIMATORS[k] for k in CORE}, CONFIG, True)                  #
cấu hình chuẩn, chỉ mua
assert all(np.allclose(BASE["out"][k]["r"].values, RESULTS["long_only"]
["out"][k]["r"].values) for k in CORE), "run ≠ §4"
U0 = BASE["ulog"]
ENV = dict(python=platform.python_version(), numpy=np.__version__,
pandas=pd.__version__, arch=__import__("arch").__version__, SEED=SEED,
**CONFIG)
json.dump(ENV, open(f"{RESULTS_DIR}/ENV_CONFIG.json", "w"), indent=1)
print(f"Dữ liệu: {P.shape[1]} mã, {P.shape[0]} phiên ({P.index[0].date()} →
{P.index[-1].date()}) | OOS: {len(U0)} kỳ, "
      f"{len(BASE['out']['EW-RIE*']['r'])} phiên ({U0[0]['date'].date()} →
{BASE['out']['EW-RIE*']['r'].index[-1].date()})")
print(f"Mã đủ điều kiện mỗi kỳ: {min(u['n_eligible'] for u in U0)}–
{max(u['n_eligible'] for u in U0)} (N={CONFIG['N']}, ZERO_MAX=
{CONFIG['ZERO_MAX']:.0%}) | "
      f"run() trùng §4: OK | môi trường: {ENV['python']}, numpy
{ENV['numpy']}, pandas {ENV['pandas']}, arch {ENV['arch']}")
```

```
Dữ liệu: 493 mã, 2957 phiên (2015-01-05 → 2026-11-10) | OOS: 118 kỳ, 2475 phiên
(2016-12-08 → 2026-11-10)
Mã đủ điều kiện mỗi kỳ: 100–315 (N=100, ZERO_MAX=20%) | run() trùng §4: OK | mô
i trường: 3.13.18, numpy 2.5.3, pandas 3.0.5, arch 8.0.0
```

T2 – Độ vững: chi phí giao dịch, tần suất tái cân bằng, phiên không giao dịch

Câu hỏi: hiệu quả của EW-RIE* có còn khi thay đổi điều kiện vận hành danh mục?

- **T2a** lưới REB ∈ {5, 21, 63} × COST ∈ {0,15%; 0,25%; 0,40%} (9 kịch bản × 2 chế độ).
- **T2b** phiên lợi suất 0 (mã đứng giá): không lọc vs lọc > 20% (**chuẩn v3**) vs lọc > 10% phiên r = 0 trong cửa sổ quá khứ.
- **T2c** quy ước phí: COST một chiều vs phí thực tế mua 0,15% / bán 0,25% (thuế bán 0,1%); unit test turnover × cost.

In [24]:
```
# ---- T2: Độ vững ----
# T2a. Lưới REB ∈ {5, 21, 63} × COST ∈ {0,15%; 0,25%; 0,40%} cho NL-LW, EW-NL-LW, EW-RIE*, cả hai chế độ.
# COST không làm đổi tỷ trọng ⇒ mỗi (REB, chế độ) chạy 1 backtest; Sharpe sau phí ở từng COST tính lại từ chuỗi turnover (kiểm ở T2c).
REBS, COSTS = (5, 21, 63), (0.0015, 0.0025, 0.004)
T2 = {(reb, m): run({k: ESTIMATORS[k] for k in CORE}, dict(CONFIG, REB=reb), lo) for reb in REBS for m, lo in MODES.items()}
T2A = pd.DataFrame([dict(mode=m, REB=reb, COST=c, method=k, **metrics(o, c), **{"Turnover/năm": o["m"]["Turnover/kỳ"] * 252 / reb})
                    for (reb, m), d in T2.items() for k, o in d["out"].items() for c in COSTS])
save(T2A, "T2a_reb_cost_grid")
g = T2A.sort_values("COST").groupby(["method", "mode", "REB"])
assert g["Sharpe sau phí"].apply(lambda s: (s.diff().dropna() <= 1e-12).all()).all(), "COST tăng mà Sharpe sau phí tăng?"

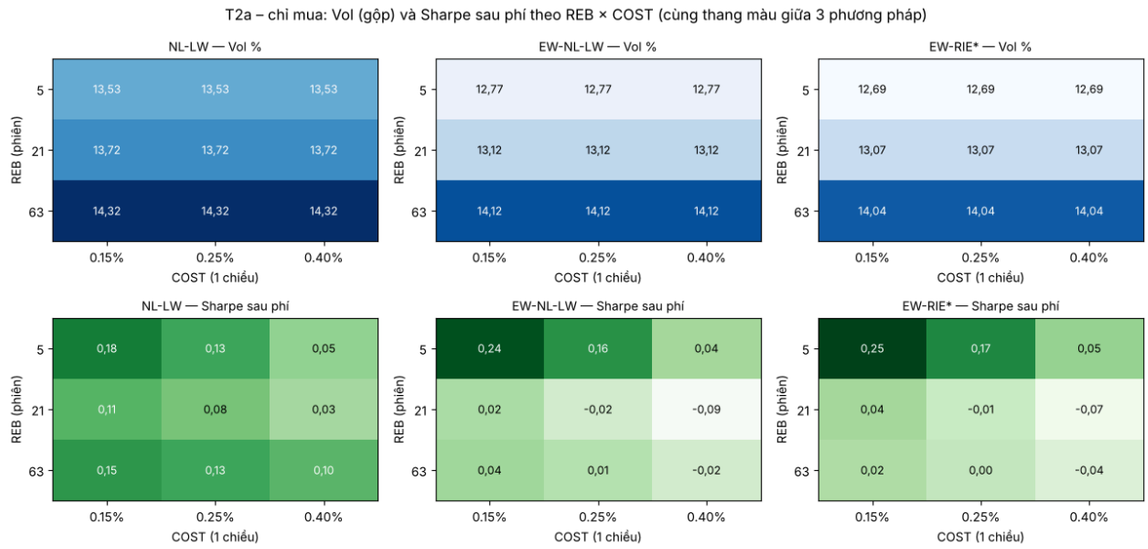
tab = T2A[T2A.COST == CONFIG["COST"]].pivot_table(index=["mode", "REB"], columns="method",
        values=["Vol %", "Sharpe sau phí", "Turnover/năm"])[["Vol %", "Sharpe sau phí", "Turnover/năm"]].reindex(columns=CORE, level=1)
print(f"Bảng T2a — COST = {CONFIG['COST']:.2%} (Vol %/năm gộp; Sharpe sau phí; turnover năm hóa = Σ|Δw|/kỳ × 252/REB)")
show(tab.rename(index=VN, level=0))

fig, ax = plt.subplots(2, 3, figsize=(13, 6.3)); LOD = T2A[T2A["mode"] == "long_only"]
for j, k in enumerate(CORE):
    for i, (col, cm) in enumerate([("Vol %", "Blues"), ("Sharpe sau phí", "Greens")]):
        M = LOD[LOD.method == k].pivot(index="REB", columns="COST", values=col); lo_, hi_ = LOD[col].min(), LOD[col].max()
        ax[i, j].imshow(M.values, cmap=cm, aspect="auto", vmin=lo_, vmax=hi_)
        ax[i, j].set_xticks(range(3), [f"{c:.2%}" for c in M.columns]); ax[i, j].set_yticks(range(3), M.index)
        for (y, x), v in np.ndenumerate(M.values):
            ax[i, j].text(x, y, f2(v), ha="center", va="center", fontsize=9, color="white" if v > (lo_ + hi_) / 2 else "black")
        ax[i, j].set_title(f"{k} — {col}", fontsize=10); ax[i, j].set_xlabel("COST (1 chiều)"); ax[i, j].set_ylabel("REB (phiên)")
fig.suptitle("T2a — chỉ mua: Vol (gộp) và Sharpe sau phí theo REB × COST (cùng thang màu giữa 3 phương pháp)"); plt.tight_layout(); savefig(fig, "T2a_heatmap")

a = T2A[T2A.COST == CONFIG["COST"]].set_index(["mode", "REB", "method"])
win = lambda m: (T2A[T2A["mode"] == m].pivot_table(index=["REB", "COST"], columns="method", values="Sharpe sau phí")
                 .pipe(lambda x: (x["NL-LW"] > x["EW-RIE*"]).sum()))
low = sum(a.loc[(m, r)]["Vol %"].idxmin() == "EW-RIE*" for m in MODES for r in REBS)
```

```
Bảng T2a — COST = 0.25% (Vol %/năm gộp; Sharpe sau phí; turnover năm hóa = Σ|Δw|/kỳ × 252/REB)
```

		Vol %			Sharpe sau phí			Turnover/năm		
	method	NL-LW	EW-NL-LW	EW-RIE*	NL-LW	EW-NL-LW	EW-RIE*	NL-LW	EW-NL-LW	EW-RIE*
mode	REB									
Chỉ mua	5	13,530	12,770	12,690	0,129	0,157	0,170	7,037	10,272	10,134
	21	13,721	13,124	13,066	0,077	-0,019	-0,008	4,115	5,832	5,726
	63	14,325	14,117	14,042	0,129	0,015	0,001	2,612	3,507	3,432
Bán khống	5	12,870	12,386	12,202	-0,096	-0,060	-0,059	18,829	22,952	21,501
	21	13,100	12,719	12,557	-0,051	-0,133	-0,143	9,748	12,056	11,313
	63	13,597	13,477	13,331	0,127	0,083	0,017	5,685	6,887	6,459



Nhận xét T2a.

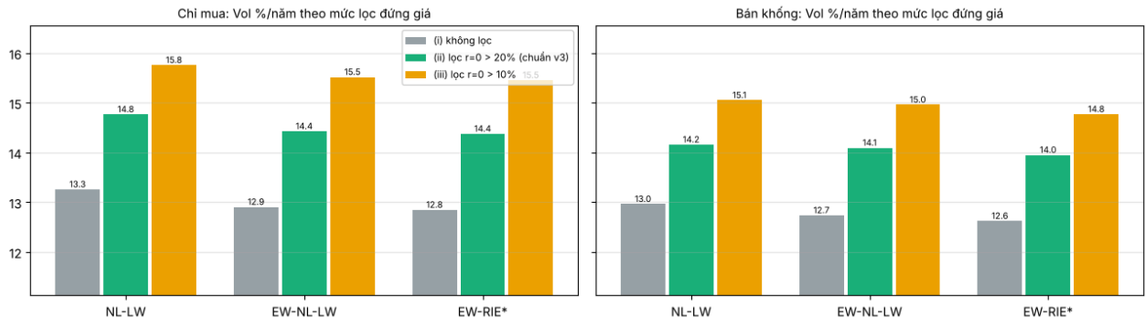
- **Rủi ro:** EW-RIE* có Vol thấp nhất trong 3 phương pháp ở **6/6 tổ hợp** (REB × chế độ).
  - Thấp hơn EW-NL-LW 0,06–0,18 pp/năm (lớn nhất ở bán khống REB = 5).
  - Thấp hơn NL-LW 0,27–0,84 pp/năm.
- **Tần suất** (chỉ mua): REB 5 → 21 → 63 làm Vol EW-RIE* tăng 12,69% → 13,07% → 14,04%, còn turnover/năm giảm 10,1 → 5,7 → 3,4. Tái cân bằng thưa làm mất lợi thế "phản ứng nhanh" của trọng số mũ: ở REB = 63, chênh với NL-LW chỉ còn 0,28 pp.
- **Sau phí (heatmap Sharpe):**
  - Ở REB = 5, EW-RIE* có Sharpe sau phí cao nhất (0,170 so với NL-LW 0,129, chỉ mua).
  - Ở REB = 21/63, NL-LW thắng (0,077 so với −0,008 ở REB = 21) vì turnover thấp hơn ≈ 30%.
- Cả 3 phương pháp đều có Sharpe sau phí thấp (≤ 0,17): bài nên trình bày đóng góp như một **bộ ước lượng rủi ro**, không phải chiến lược sinh lời.

In [25]:
```
# T2b. Phiên lợi suất 0. Code gốc: phiên không khớp lệnh vẫn có giá ⇒ r = 0; NaN trong cửa sổ → fillna(0).
# So sánh (i) gốc với (ii) loại khỏi universe các mã có tỷ lệ r=0 trong CỬA SỔ QUÁ KHỨ > 20% hoặc > 10% (không dùng tương lai).
def zero_share(ulog):    # tỷ lệ r=0 trung bình của universe đã chọn (đúng dữ liệu đưa vào estimator)
    return np.mean([(R_BT.iloc[POS[u["date"]] - CONFIG["WINDOW"]:POS[u["date"]]][u["tickers"]].fillna(0) == 0).values.mean() for u in ulog])
VARS = {"(i) không lọc": 1.0, "(ii) lọc r=0 > 20% (chuẩn v3)": 0.20, "(iii) lọc r=0 > 10%": 0.10}
T2B_RUN = {(v, m): run({k: ESTIMATORS[k] for k in CORE}, dict(CONFIG, ZERO_MAX=z), lo)
           for v, z in VARS.items() for m, lo in MODES.items()}
COMMON = sorted(set.intersection(*[set(d["out"]["NL-LW"]["r"].index) for d in T2B_RUN.values()]))   # kỳ thiếu mã bị bỏ ⇒ so trên ngày chung
T2B = pd.DataFrame([dict(variant=v, mode=m, method=k, **metrics(o, dates=COMMON), **{"Số kỳ hợp lệ": len(d["ulog"]),
                    "Mã đủ ĐK (TB)": np.mean([u["n_eligible"] for u in d["ulog"]]), "Mã đủ ĐK (min)": min(u["n_eligible"] for u in d["ulog"]),
                    "% r=0 trong universe": 100 * zero_share(d["ulog"])}) for (v, m), d in T2B_RUN.items() for k, o in d["out"].items()])
save(T2B, "T2b_zero_return")
print(f"Bảng T2b — Vol %/năm trên {len(COMMON)} ngày OOS chung ({COMMON[0].date()} → {COMMON[-1].date()}); tổng số kỳ chuẩn: {len(U0)}")
tb = T2B.pivot_table(index="variant", columns=["mode", "method"], values="Vol %").reindex(columns=CORE, level=1)
info = T2B[T2B["mode"] == "long_only"].groupby("variant")[["Số kỳ hợp lệ", "Mã đủ ĐK (TB)", "Mã đủ ĐK (min)", "% r=0 trong universe"]].first()
show(pd.concat([tb.rename(columns=VN, level=0), pd.concat({"Universe": info}, axis=1)], axis=1).loc[list(VARS)], 2)
z = tb["long_only"]; base = z.loc[list(VARS)[0]]

fig, ax = plt.subplots(1, 2, figsize=(13, 3.8), sharey=True)
for a, m in zip(ax, MODES):
    d = tb[m].loc[list(VARS)]; x = np.arange(len(CORE))
    for j, (vn, col) in enumerate(zip(VARS, ["#9aa0a6", "#1baf7a", "#eda100"])):
        a.bar(x + (j - 1) * .27, d.loc[vn, CORE], .25, color=col, label=vn)
        for i, v in enumerate(d.loc[vn, CORE]): a.text(i + (j - 1) * .27, v, f"{v:.1f}", ha="center", va="bottom", fontsize=7)
    a.set_xticks(x, CORE); a.set_ylim(tb.values.min() - 1.5, tb.values.max() + .8); a.set_title(f"{VN[m]}: Vol %/năm theo mức lọc đứng giá", fontsize=10); a.grid(axis="y", alpha=.3)
ax[0].legend(fontsize=8); plt.tight_layout(); savefig(fig, "T2b_zero_filter")
```

```
Bảng T2b — Vol %/năm trên 1446 ngày OOS chung (2021-01-20 → 2026-11-10); tổng số kỳ chuẩn: 118
```

	Chỉ mua			Bán khống					Universe
	NL-LW	EW-NL-LW	EW-RIE*	NL-LW	EW-NL-LW	EW-RIE*	Số kỳ hợp lệ	Mã đủ ĐK (TB)	Mã đủ ĐK (min)	% r=0 trong universe
variant										
(i) không lọc	13,26	12,91	12,85	12,97	12,74	12,63	129	263,90	166	10,10
(ii) lọc r=0 > 20% (chuẩn v3)	14,77	14,43	14,38	14,16	14,10	13,95	118	232,57	100	8,55
(iii) lọc r=0 > 10%	15,76	15,52	15,46	15,05	14,96	14,77	69	161,45	108	6,13



Nhận xét T2b – lọc mã đứng giá.

- **Mức rủi ro:** lọc mã nhiều phiên r = 0 làm Vol **tăng** đồng loạt ở cả 3 phương pháp và 2 chế độ. Trên 1.446 ngày chung (2021–2026), Vol chỉ mua của EW-RIE* là 12,85% (không lọc) → 14,38% (20%) → 15,46% (10%). Tỷ lệ r = 0 trong universe giảm 10,1% → 8,6% → 6,1%.
- **Cơ chế:** mã đứng giá trông như ít biến động và ít tương quan nên GMV dồn tỷ trọng vào đó, rồi đo rủi ro ngoài mẫu cũng trên chính các chuỗi giá "đứng". Vì thế Vol không lọc **đánh giá thấp rủi ro thật**. Đây là lý do v3 lấy ZERO_MAX = 20% làm chuẩn.
- **Thứ hạng giữ nguyên** ở mọi mức lọc: EW-RIE* < EW-NL-LW < NL-LW.
- **Chọn ngưỡng:** 20% giữ 118/129 kỳ. Ngưỡng 10% chỉ còn 69 kỳ đủ N = 100 và cho Sharpe sau phí âm (−0,29 với EW-RIE*), nên không dùng làm chuẩn.

In [26]:
```
# T2c. Quy ước chi phí. run_backtest: phí kỳ = COST × Σ|w_mới − w_cũ| ⇒ COST
là phí MỘT CHIÊ`U áp cho ca̰ mua lẫn bán.
# HOSE: phí môi giới ~0,15% mỗi chiê`u + thuê´ bán 0,1% ⇒ bâ´t đô´i xứng
(mua 0,15%, bán 0,25%).
def trade_cost(w_new, w_old, c_buy, c_sell):
    u = w_new.index.union(w_old.index); d = w_new.reindex(u, fill_value=0) -
w_old.reindex(u, fill_value=0)
    return c_buy * d.clip(lower=0).sum() + c_sell * (-d).clip(lower=0).sum()
rng = np.random.default_rng(SEED)
for _ in range(200):     # unit test: có mã ra/vào universe và ty̰ trọng âm
(bán khô´ng), Σw = 1 ở ca̰ hai kỳ
    a_ = pd.Series(rng.normal(size=6), index=list("ABCDEF")); a_ /= a_.sum();
b_ = pd.Series(rng.dirichlet(np.ones(4)), index=list("DEFG"))
    u = a_.index.union(b_.index); turn = (a_.reindex(u, fill_value=0) -
b_.reindex(u, fill_value=0)).abs().sum()
    assert np.isclose(trade_cost(a_, b_, .0025, .0025), .0025 * turn) and
np.isclose(trade_cost(a_, b_, .0015, .0025), .002 * turn)
o_new = run({"NL-LW": est_nllw}, dict(CONFIG, REB=63, COST=0.004))["out"]
["NL-LW"]          # chạy lại thật với COST khác
assert np.allclose(net(T2[63, "long_only"]["out"]["NL-LW"], 0.004).values,
o_new["r_net"].values, atol=1e-14)
print("Unit test 200/200 pass: Σw=1 ⇒ giá trị mua = giá trị bán = turnover/2
⇒ phí (mua 0,15%, bán 0,25%) ≡ COST đối xứng 0,20%.")
print("Kiểm chứng: r_net tính lại từ turnover trùng với chạy lại backtest
thật (REB=63, COST=0,40%).")
SC = {"Gộp": None, "COST 0,25% (gốc)": .0025, "Mua 0,15% / bán 0,25%": .002}
T2C = pd.DataFrame([dict(mode=VN[m], method=k, scen=sc, Sharpe=ann(o["r"] if
c is None else net(o, c))[1])
                    for m in MODES for k, o in T2[21, m]["out"].items() for
sc, c in SC.items()]).pivot(index="method", columns=["mode", "scen"],
values="Sharpe")
T2C = T2C.reindex(index=CORE,
columns=pd.MultiIndex.from_product([VN.values(), SC]))
save(T2C, "T2c_cost_convention"); print("Sharpe (REB = 21):"); show(T2C)
```

```
Unit test 200/200 pass: Σw=1 ⇒ giá trị mua = giá trị bán = turnover/2 ⇒ phí (mu
a 0,15%, bán 0,25%) ≡ COST đối xứng 0,20%.
Kiểm chứng: r_net tính lại từ turnover trùng với chạy lại backtest thật (REB=6
3, COST=0,40%).
Sharpe (REB = 21):
```

	Chỉ mua			Bán khống		
	Gộp	COST 0,25% (gốc)	Mua 0,15% / bán 0,25%	Gộp	COST 0,25% (gốc)	Mua 0,15% / bán 0,25%
method						
NL-LW	0,152	0,077	0,092	0,134	-0,051	-0,014
EW-NL-LW	0,091	-0,019	0,003	0,102	-0,133	-0,086
EW-RIE*	0,101	-0,008	0,014	0,080	-0,143	-0,099

Nhận xét T2c. Với Σw = 1, giá trị mua bằng giá trị bán nên phí thực tế bất đối xứng (mua 0,15% / bán 0,25%) **tương đương COST đối xứng 0,20%**. Unit test và lần chạy lại thật đều xác nhận điều này. Ở mức phí thực tế, Sharpe sau phí (REB = 21, chỉ mua) là: EW-RIE* 0,014 (so với −0,008 ở giả định gốc 0,25%), EW-NL-LW 0,003, NL-LW 0,092. Giả định 0,25% hơi bảo thủ nhưng không đổi kết luận: sau phí NL-LW vẫn hơn ở REB = 21. Hạn chế còn lại: lợi suất kỳ tính bằng tỷ trọng cố định `Y @ w` (không mô phỏng trôi tỷ trọng), chưa có spread/trượt giá, kỳ đầu không trừ phí mua.

T3 – Đối chứng & kiểm định

Câu hỏi: EW-RIE* có vượt các đối chứng mạnh trong cùng điều kiện, và chênh lệch có vượt dao động ngẫu nhiên của mẫu?

- **T3a** đối chứng mới: **EW-WeSpeR (v3)**, 1/N (`est_equal_weight_proxy` , chiến lược phân bổ – không phải bộ ước lượng), PCA k nhân tố (k ∈ {1,3,5}; PCA-auto chọn k chỉ trong cửa sổ huấn luyện), DCC-GARCH(1,1) (GARCH đơn biến bằng `arch` + DCC đầy đủ N×N, (a, b) ước lượng bằng composite likelihood).
- **T3b** bảng benchmark: Vol, Sharpe trước/sau phí, turnover, số mã hiệu dụng, thời gian chạy; Vol theo giai đoạn.
- **T3c** kiểm định phương sai Ledoit–Wolf (2011): $\Delta = \log \sigma^2(\text{EW-RIE*}) - \log \sigma^2(\text{đối chứng})$, HAC và studentized circular block bootstrap (block 10/21/42); Sharpe: Jobson–Korkie–Memmel; Holm cho 5 so sánh mỗi chế độ.
- **T3d** hình: Vol kèm CI 95%, heatmap p-value.

In [27]:

```
# ---- T3: Đố́i chứng & kiể̉m định ----
# T3a. Đố́i chứng mới: 1/N, mô hình nhân tố PCA (k ∈ {1,3,5} và tự chọn k
trong cử̉a sổ̉ huấ́n luyện), DCC-GARCH(1,1).
from arch import arch_model
from scipy.optimize import minimize

def est_equal_weight_proxy(X):
    # 1/N: dùng ma trận đơn vị -> GMV = 1/N (chiế́n lược phân bổ̀, KHÔNG
phả̉i bộ ước lượng hiệp phương sai)
    return np.eye(X.shape[1])

def est_pca(k):
    # Σ = B Bᵀ + diag(ψ): k thành phầ̀n chính cử̉a hiệp phương sai mẫ̃u,
phầ̀n dư chéo (đố́i xứng, PSD theo cấ́u trúc)
    def est(X):
        S = np.cov(X, rowvar=False); lam, Q = np.linalg.eigh(S); B = Q[:, -
k:] * np.sqrt(lam[-k:])
        return B @ B.T + np.diag(np.maximum(np.diag(S) - (B**2).sum(1), 1e-
10))
    return est

PCA_K = []
def est_pca_auto(X, ks=(1, 3, 5), split=0.7):
    # chọn k CHỈ trong cử̉a sổ̉ huấ́n luyện: ước lượng trên 70% đầ̀u, đo
phương sai GMV trên 30% cuố́i, rồ̀i ước lượng lại trên cả̉ cử̉a sổ̉
    n = int(len(X) * split)
    k = min(ks, key=lambda k: np.var(X[n:] @ gmv_unconstrained(est_pca(k)
(X[:n]))))
    PCA_K.append(k); return est_pca(k)(X)

DCC_LOG, _DCC_CACHE = [], {}
def _dcc_ncl(p, Z, Qbar):
    # −composite log-lik (Engle, Shephard & Sheppard 2008): tổ̉ng log-lik 2×2
cử̉a các cặp kề̀ nhau (i, i+1).
    # Full QML với N=100 vừa chậm vừa kéo a → 0 (thiên lệch đã biế́t khi N
lớn) nên dùng CL.
    a, b = p
    if a < 0 or b < 0 or a + b >= 0.999: return 1e10
    z1, z2 = Z[:, :-1], Z[:, 1:]; q11, q22, q12 = np.diag(Qbar)[:-1],
np.diag(Qbar)[1:], np.diag(Qbar, 1)
    c11, c22, c12 = q11.copy(), q22.copy(), q12.copy(); nll = 0.0
    for t in range(len(Z)):
        if t:
            c11 = (1-a-b)*q11 + a*z1[t-1]**2 + b*c11; c22 = (1-a-b)*q22 +
a*z2[t-1]**2 + b*c22
            c12 = (1-a-b)*q12 + a*z1[t-1]*z2[t-1] + b*c12
        rho = c12 / np.sqrt(c11 * c22); d = 1 - rho**2
        nll += 0.5 * np.sum(np.log(d) + (z1[t]**2 + z2[t]**2 -
2*rho*z1[t]*z2[t]) / d)
    return nll

def est_dcc(X, h=21):
    # DCC-GARCH(1,1) (Engle 2002): bước 1 GARCH(1,1) đơn biế́n (`arch`) cho
từng mã; bước 2 (a, b) bằng composite
    # likelihood + correlation targeting; ma trận Q_t đầ̀y đủ̉ N×N. Dự báo: D
= căn phương sai GARCH trung bình h phiên tới, R = R_{T+1|T}; Σ = D R D.
    key = hash(X.tobytes())
    if key in _DCC_CACHE: return _DCC_CACHE[key]                # cùng cử̉a
sổ̉ ở long_only/short_ok → không ước lượng lại
    T, N = X.shape; Z = np.empty_like(X); hv = np.empty(N); bad = 0
    for i in range(N):
        try:
            r = arch_model(100 * X[:, i], mean="Constant", vol="GARCH", p=1,
q=1, rescale=False).fit(disp="off", show_warning=False)
            ok = r.convergence_flag == 0 and r.params["alpha[1]"] +
r.params["beta[1]"] < 1
        except Exception:
            ok = False
        if ok:
            Z[:, i] = r.std_resid; hv[i] = r.forecast(horizon=h,
reindex=False).variance.values[-1].mean() / 1e4
        else:                                                   # dự phòng:
```

```
phương sai mẫu (đệm vào DCC_LOG)
            bad += 1; Z[:, i] = (X[:, i] - X[:, i].mean()) / X[:, i].std(); hv[i] = X[:, i].var()
    Qbar = Z.T @ Z / T
    res = minimize(_dcc_ncl, x0=[0.02, 0.95], args=(Z, Qbar), method="Nelder-Mead", options=dict(xatol=1e-4, fatol=1e-3, maxiter=200))
    a, b = res.x if res.fun < 1e9 else (0.0, 0.0)
    Qt = Qbar.copy()
    for t in range(1, T + 1): Qt = (1 - a - b) * Qbar + a * np.outer(Z[t-1], Z[t-1]) + b * Qt    # Q_{T+1}
    d = 1 / np.sqrt(np.diag(Qt)); sd = np.sqrt(hv)
    S = (Qt * np.outer(d, d)) * np.outer(sd, sd)
    DCC_LOG.append(dict(a=a, b=b, garch_fail=bad, dcc_ok=bool(res.success))); _DCC_CACHE[key] = S
    return S
```

In [28]:
```
# T3b. Baʾng benchmark: cùng ngày OOS, cùng universe, REB=21, COST=0,25%, hai chêˊ độ.
T3_EST = {"1/N": est_equal_weight_proxy, **{f"PCA-k{k}": est_pca(k) for k in (1, 3, 5)}, "PCA-auto": est_pca_auto,
          "DCC-GARCH(1,1)": est_dcc, "EW-WeSpeR": ESTIMATORS["EW-WeSpeR"], **{k: ESTIMATORS[k] for k in CORE}}
T3R = {m: run(T3_EST, CONFIG, lo) for m, lo in MODES.items()}
T3B = pd.DataFrame([dict(mode=m, method=k, **metrics(o), **{"Tỷ trọng max %": o["m"]["Tỷ trọng max %"], "Giây/lần ước lượng": T3R["long_only"]["sec"][k]})
                    for m, d in T3R.items() for k, o in d["out"].items()])
save(T3B, "T3b_benchmark")
for m in MODES: assert T3B[T3B["mode"] == m].set_index("method")["Turnover/kỳ"].idxmin() == "1/N"   # sanity: 1/N ít giao dịch nhâˊt
tb = T3B.pivot(index="method", columns="mode", values=["Vol %", "Sharpe", "Sharpe sau phí", "Turnover/kỳ", "Sôˊ mã hiệu dụng"])
tb.columns = tb.columns.swaplevel(); tb = tb[[c for m in MODES for c in tb.columns if c[0] == m]].sort_values(("long_only", "Vol %"))
tb["Giây/lần ước lượng"] = T3B.drop_duplicates("method").set_index("method")["Giây/lần ước lượng"]
print("Bảng T3b — benchmark (xếp theo Vol chỉ mua; Vol %/năm gộp; Sharpe sau phí trừ COST × turnover)"); show(tb.rename(columns=VN, level=0))
P3 = pd.concat({VN[m]: summarize(d["out"], [])[1] for m, d in T3R.items()}); save(P3, "T3b_periods")
print("Vol %/năm theo giai đoạn thị trường:"); show(P3, 2)
if len(DCC_LOG) >= len(U0):   # log chiʾ có khi vừa tính mới (không dùng cache) → lưu CSV; kỳ đầu tiên cuʾa chêˊ độ chiʾ mua

pd.DataFrame(DCC_LOG[:len(U0)]).assign(pca_auto_k=PCA_K[:len(U0)]).to_csv(f"{RESULTS_DIR}/T3_dcc_pca_log.csv", index=False)
dl = pd.read_csv(f"{RESULTS_DIR}/T3_dcc_pca_log.csv"); pk = dl.pca_auto_k
v = tb[("long_only", "Vol %")]
```

```
Bảng T3b — benchmark (xếp theo Vol chỉ mua; Vol %/năm gộp; Sharpe sau phí trừ COST × turnover)
```

mode	Chỉ mua							
	Vol %	Sharpe	Sharpe sau phí	Turnover/kỳ	Số mã hiệu dụng	Vol %	Sharpe	Sharpe sau phí	Turnov
method									
EW-WeSpeR	13,054	0,107	-0,001	0,475	9,394	12,568	0,138	-0,081	
EW-RIE*	13,066	0,101	-0,008	0,477	9,455	12,557	0,080	-0,143	
EW-NL-LW	13,124	0,091	-0,019	0,486	9,166	12,719	0,102	-0,133	
PCA-k3	13,579	0,239	0,165	0,339	9,906	14,081	0,240	0,113	
PCA-auto	13,631	0,232	0,153	0,359	9,901	13,844	0,285	0,113	
PCA-k1	13,639	0,237	0,167	0,324	10,414	14,589	0,418	0,327	
PCA-k5	13,701	0,226	0,149	0,356	9,677	13,705	0,223	0,062	
NL-LW	13,721	0,152	0,077	0,343	14,396	13,100	0,134	-0,051	
DCC-GARCH(1,1)	13,950	0,344	0,155	0,885	8,115	15,168	0,385	-0,187	
1/N	22,733	0,424	0,416	0,054	100,000	22,733	0,424	0,416	

```
Vol %/năm theo giai đoạn thị trường:
```

		2016-17 Ổn định	2018-19 Điều chỉnh	2020Q1 COVID	2020-21 Bùng nổ	2022 Sụt giảm	2023-24 Phục hồi	2025-26
Chỉ mua	1/N	11,34	16,04	32,45	23,51	36,66	21,70	21,91
	PCA-k1	9,60	11,97	16,89	13,04	18,64	11,24	15,80
	PCA-k3	9,73	11,87	17,05	13,00	17,97	11,42	15,81
	PCA-k5	9,74	11,74	17,10	13,37	18,31	11,39	15,93
	PCA-auto	9,64	11,76	17,01	13,22	18,24	11,45	15,79
	DCC-GARCH(1,1)	9,89	12,74	16,98	12,88	17,90	11,95	16,46
	EW-WeSpeR	8,77	11,30	16,13	11,66	17,77	11,10	15,66
	NL-LW	9,52	12,10	20,67	13,08	18,11	11,42	15,50
	EW-NL-LW	8,84	11,45	16,14	11,79	17,77	11,08	15,77
	EW-RIE*	8,81	11,33	16,11	11,73	17,75	11,02	15,71
Bán khống	1/N	11,34	16,04	32,45	23,51	36,66	21,70	21,91
	PCA-k1	9,61	10,90	18,62	13,12	21,03	11,55	18,90
	PCA-k3	9,49	10,77	17,94	13,43	18,82	11,28	18,23
	PCA-k5	9,55	10,83	17,99	13,13	18,78	11,01	16,92
	PCA-auto	9,66	10,65	18,01	13,27	19,19	11,26	17,12
	DCC-GARCH(1,1)	11,17	12,67	18,93	14,73	18,13	13,44	18,36
	EW-WeSpeR	8,50	10,08	16,47	11,52	17,13	10,03	15,83
	NL-LW	9,38	10,87	20,48	12,81	16,95	10,66	15,20
	EW-NL-LW	8,78	10,42	16,65	11,60	17,25	9,97	16,03
	EW-RIE*	8,59	10,14	16,45	11,48	17,12	9,90	15,81

Nhận xét T3b.

- **Chỉ mua:** thứ tự Vol là **EW-WeSpeR 13,05% ≈ EW-RIE* 13,07%** < EW-NL-LW 13,12% < PCA-k3 13,58% ≈ PCA-auto 13,63% < NL-LW 13,72% < DCC-GARCH 13,95% ≪ 1/N 22,73%.
- **Bán khống:** EW-RIE* thấp nhất (12,56%), EW-WeSpeR 12,57%, EW-NL-LW 12,72%. Mô hình nhân tố PCA và DCC (13,7–15,2%) kém hơn cả NL-LW (13,10%).
- **Theo giai đoạn:** nhóm EW thắng rõ nhất trong COVID (16,1% so với NL-LW 20,7%) và 2020–21. Năm 2025–26 NL-LW thấp hơn một chút (15,50% so với 15,7%).
- **Sau phí:** Sharpe cao nhất thuộc về 1/N (0,416, turnover 0,05) và PCA (≈ 0,15–0,17), nhờ turnover thấp. Nhóm EW có Sharpe sau phí ≈ 0.
- **DCC-GARCH(1,1):** a TB 0,027, b TB 0,71. Có 277/11.800 lần fit GARCH không hội tụ (dùng phương sai mẫu thay thế). Turnover cao nhất (0,89 chỉ mua, 2,92 bán khống).
- **PCA-auto** chọn k = 1/3/5 lần lượt 39/31/48 kỳ.
- **Lưu ý cột "giây/lần":** EW-WeSpeR hiển thị 0,001 s vì dùng lại cache ước lượng từ §4. Thời gian thật ≈ 1–4 s/lần (T6, cấu hình không cache), chậm hơn EW-RIE* (0,16 s) khoảng 10–25 lần.

In [29]:

```
# T3c. Kiểm định: phương sai Ledoit–Wolf (2011) — HAC và studentized
circular block bootstrap; Sharpe Jobson–Korkie–Memmel;
# so với block bootstrap CI có sẵn (block_bootstrap_ci), block ∈ {10, 21,
42}; Holm cho 4 so sánh mỗi chế độ.
def hac(u, L=None):
    # HAC Newey–West (Bartlett), L = ⌊4(T/100)^(2/9)⌋. (LW 2008/2011 dùng
QS/Parzen + prewhitening — chưa làm.)
    T = len(u); L = L or int(4 * (T / 100) ** (2 / 9)); S = u.T @ u / T
    for l in range(1, L + 1): G = u[l:].T @ u[:-l] / T; S += (1 - l / (L +
1)) * (G + G.T)
    return S

def _lw_stat(y, Psi=None, b=None):
    # Δ = log σ²_a − log σ²_b (LW 2011, delta method); y_t = (r_a, r_b, r_a²,
r_b²)
    T = len(y); m = y.mean(0); va, vb = m[2] - m[0]**2, m[3] - m[1]**2
    g = np.array([-2 * m[0] / va, 2 * m[1] / vb, 1 / va, -1 / vb])
    if Psi is None:   # Ψ của bootstrap theo khối (LW 2008, công thức (9))
        z = (y - m)[:T // b * b].reshape(-1, b, 4).sum(1) / np.sqrt(b); Psi =
z.T @ z / len(z)
    return np.log(va) - np.log(vb), np.sqrt(g @ Psi @ g / T)

def lw_var_test(ra, rb, block=21, M=2000, seed=SEED):
    # p-value HAC (chuẩn) và p-value studentized circular block bootstrap
(LW 2008 Remark 3.2, áp cho phương sai như LW 2011)
    y = np.column_stack([ra, rb, ra**2, rb**2]); T = len(y)
    d, se = _lw_stat(y, hac(y - y.mean(0))); p_hac = 2 * norm.sf(abs(d / se))
    rng = np.random.default_rng(seed); l = T // block; stats = np.empty(M)
    for i in range(M):
        ix = ((rng.integers(0, T, l)[:, None] + np.arange(block)) %
T).ravel()      # khối vòng
        ds, ses = _lw_stat(y[ix], b=block); stats[i] = abs(ds - d) / ses
    return d, se, p_hac, (np.sum(stats >= abs(d) / se) + 1) / (M + 1)

def jkm_test(ra, rb):
    # Jobson–Korkie (1981) với hiệu chỉnh Memmel (2003); giả định iid
chuẩn
    T = len(ra); m1, m2 = ra.mean(), rb.mean(); s1, s2 = ra.std(ddof=1),
rb.std(ddof=1); s12 = np.cov(ra, rb)[0, 1]
    th = (2 * s1**2 * s2**2 - 2 * s1 * s2 * s12 + .5 * m1**2 * s2**2 + .5 *
m2**2 * s1**2 - m1 * m2 / (s1 * s2) * s12**2) / T
    z = (s2 * m1 - s1 * m2) / np.sqrt(th); return z, 2 * norm.sf(abs(z))

# unit test nhanh: gần trùng ⇒ p lớn; gấp đôi biến động ⇒ p≈0; JKM phản
đối xứng, Sharpe chênh lớn ⇒ p≈0
# (JKM suy biến θ=0 khi 2 chuỗi tương quan hoàn hảo — không dùng trường
hợp đó)
_r = np.random.default_rng(SEED).standard_normal(1500) * .01
assert lw_var_test(_r, _r + 1e-5 *
np.random.default_rng(1).standard_normal(1500), M=200)[3] > .1
assert lw_var_test(_r, 2 * _r[::-1], M=200)[2] < 1e-6
# Δ = −log 4
_q = np.random.default_rng(2).standard_normal(1500) * .01
assert np.isclose(jkm_test(_r, _q)[0], -jkm_test(_q, _r)[0]) and jkm_test(_r
+ .003, _q - .003)[1] < 1e-6

BLOCKS, rows = (10, 21, 42), []
for mode in MODES:
    out = T3R[mode]["out"]
    for b_ in ("NL-LW", "EW-NL-LW", "EW-WeSpeR", "1/N", "PCA-auto"):   # đối
chứng định trước, không chọn hậu kiểm
        ra, rb = out["EW-RIE*"]["r"].values, out[b_]["r"].values
        na, nb = (net(out[k], CONFIG["COST"]).values for k in ("EW-RIE*",
b_))
        row = dict(mode=mode, pair=f"EW-RIE* vs {b_}", **{"ΔVol (pp)":
(ra.std() - rb.std()) * np.sqrt(252) * 100})
        for bl in BLOCKS:
            d, se, p_hac, p_bt = lw_var_test(ra, rb, block=bl)
            lo_, hi_ = block_bootstrap_ci(ra, rb, block=bl)
            row.update({"Δlog σ²": d, "p LW-HAC": p_hac, f"p LW-boot b={bl}":
p_bt, f"CI lo b={bl}": lo_, f"CI hi b={bl}": hi_})
        row.update({"p JKM gross": jkm_test(ra, rb)[1], "p JKM sau phí":
```

```
jkm_test(na, nb)[1],
                    "ΔSharpe gross": ann(pd.Series(ra))[1] -
ann(pd.Series(rb))[1]})
        rows.append(row)
T3C = pd.DataFrame(rows)
# multiple comparisons: Holm trên 5 so sánh trong mô˜i chê´ độ (p LW-boot
b=21)
T3C["p Holm (LW-boot b=21)"] = T3C.groupby("mode")["p LW-boot
b=21"].transform(
    lambda p: np.minimum(1, np.maximum.accumulate(np.sort(p.values) * (len(p)
- np.arange(len(p))))[np.argsort(np.argsort(p.values))])
save(T3C, "T3c_tests")
cols = ["ΔVol (pp)", "CI lo b=21", "CI hi b=21", "p LW-HAC", "p LW-boot
b=10", "p LW-boot b=21", "p LW-boot b=42", "p Holm (LW-boot b=21)", "p JKM
gross", "p JKM sau phí"]
print("Bảng T3c — EW-RIE* so với đối chứng (ΔVol < 0: EW-RIE* rủi ro thấp
hơn; CI = block bootstrap percentile b=21)")
show(T3C.assign(mode=T3C["mode"].map(VN)).set_index(["mode", "pair"])[cols],
4)
t3 = lambda m, b: T3C[(T3C["mode"] == m) & (T3C.pair == f"EW-RIE* vs
{b}")].iloc[0]
line = lambda m: "; ".join(f"{b}: {f2(t3(m, b)['ΔVol (pp)'])} pp, p Holm
{pv(t3(m, b)['p Holm (LW-boot b=21)'])}" for b in ("NL-LW", "EW-NL-LW", "PCA-
auto", "1/N"))
sig = lambda b: all(t3(m, b)["p Holm (LW-boot b=21)"] < .05 for m in MODES)
```

```
Bảng T3c — EW-RIE* so với đối chứng (ΔVol < 0: EW-RIE* rủi ro thấp hơn; CI = bl
ock bootstrap percentile b=21)
```

mode	pair	ΔVol (pp)	CI lo b=21	CI hi b=21	p LW-HAC	p LW-boot b=10	p LW-boot b=21	p LW-boot b=42	p Holm (LW-boot b=21)	p JK gro
Chỉ mua	EW-RIE* vs NL-LW	-0,6548	-1,0301	-0,2808	0,0001	0,0010	0,0010	0,0010	0,0040	0,62
	EW-RIE* vs EW-NL-LW	-0,0575	-0,0908	-0,0251	0,0004	0,0020	0,0010	0,0025	0,0040	0,50
	EW-RIE* vs EW-WeSpeR	0,0129	-0,0214	0,0501	0,4691	0,4823	0,4983	0,5002	0,4983	0,59
	EW-RIE* vs 1/N	-9,6646	-11,1902	-8,2129	0,0000	0,0005	0,0005	0,0005	0,0025	0,20
	EW-RIE* vs PCA-auto	-0,5647	-0,8971	-0,2666	0,0001	0,0010	0,0020	0,0015	0,0040	0,1
Bán khống	EW-RIE* vs NL-LW	-0,5438	-0,9896	-0,0581	0,0119	0,0230	0,0235	0,0320	0,0470	0,67
	EW-RIE* vs EW-NL-LW	-0,1623	-0,2372	-0,0935	0,0000	0,0005	0,0005	0,0010	0,0025	0,54
	EW-RIE* vs EW-WeSpeR	-0,0119	-0,0904	0,0727	0,8132	0,8241	0,8356	0,8141	0,8356	0,18
	EW-RIE* vs 1/N	-10,1745	-11,7294	-8,6278	0,0000	0,0005	0,0005	0,0005	0,0025	0,30
	EW-RIE* vs PCA-auto	-1,2873	-1,7350	-0,8137	0,0000	0,0005	0,0005	0,0005	0,0025	0,19

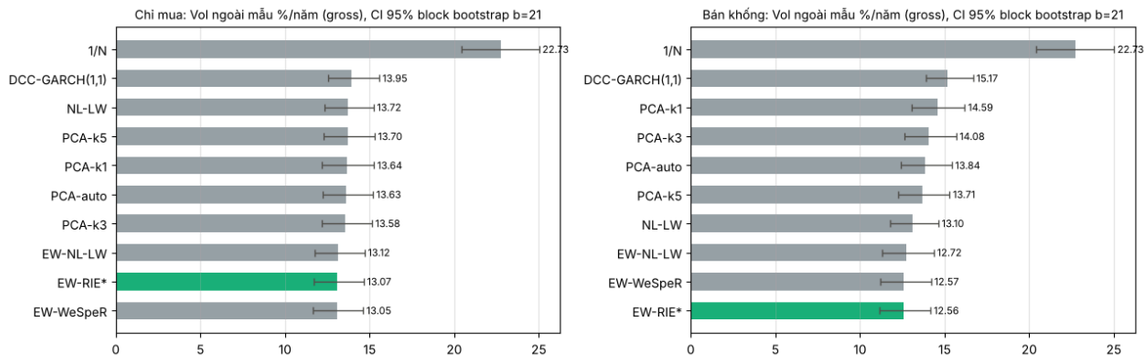
Nhận xét T3c.

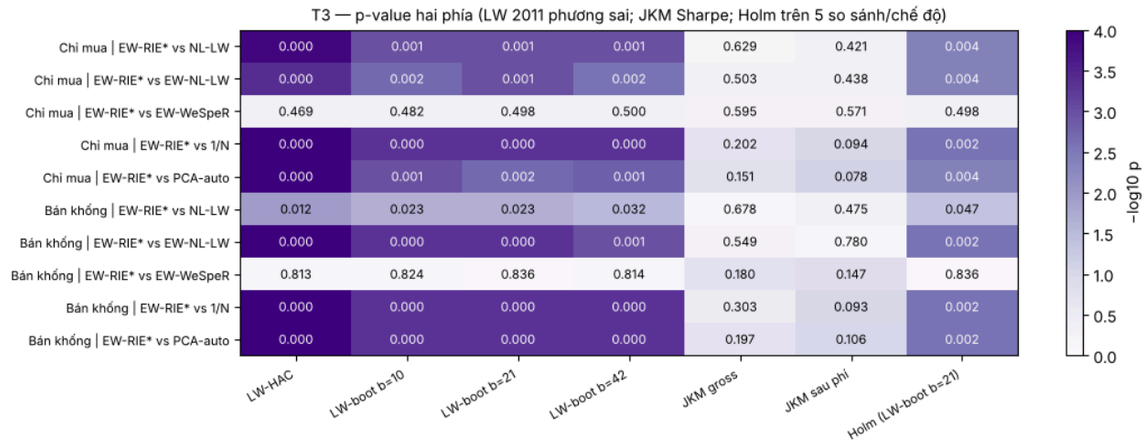
- **Chỉ mua.** EW-RIE* giảm rủi ro **có ý nghĩa sau Holm (5 so sánh)** so với:
  - NL-LW: −0,65 pp, p Holm 0,004
  - EW-NL-LW: −0,06 pp, p Holm 0,004
  - PCA-auto: −0,56 pp, p Holm 0,004
  - 1/N: −9,7 pp, p Holm 0,003
- **Bán khống.** Kết luận tương tự. Với NL-LW chỉ vừa qua ngưỡng (−0,54 pp, p Holm 0,047); với EW-NL-LW −0,16 pp, p Holm 0,003.
- **EW-RIE* vs EW-WeSpeR: không khác biệt** ở cả hai chế độ (+0,01 / −0,01 pp; p LW-boot 0,50 / 0,84).
- **Độ vững:** kết luận không đổi khi đổi block 10/21/42, và khớp với p HAC.
- **Sharpe:** không có chênh lệch nào có ý nghĩa, trước hay sau phí (JKM p ≥ 0,08).
- **Hàm ý cho bài:** đóng góp của EW-RIE* so với tài liệu mới (WeSpeR, Oriol 2024) **không nằm ở rủi ro ngoài mẫu**. Lợi thế của nó là công thức đóng, nhanh hơn ≈ 10–25 lần. Bất lợi là kém ổn định khi q_eff > 2 (T6, h = 15).

In [30]:

```
# T3d. Hình — Vol với CI 95% (block bootstrap b=21 của từng chuỗi, tái dùng
block_bootstrap_ci với chuỗi 0) + heatmap p-value ----
fig, ax = plt.subplots(1, 2, figsize=(13, 4.2), sharex=True)
for a, mode in zip(ax, MODES):
    out = T3R[mode]["out"]; v = pd.Series({k: o["m"]["Vol %"] for k, o in
out.items()}).sort_values()
    ci = np.array([block_bootstrap_ci(out[k]["r"].values, np.zeros(len(out[k]
["r"]))) for k in v.index])
    a.barh(v.index, v.values, xerr=[v.values - ci[:, 0], ci[:, 1] -
v.values], capsize=3, height=.6,
           color=[COL["EW-RIE*"] if k == "EW-RIE*" else "#9aa0a6" for k in
v.index], error_kw=dict(lw=1, ecolor="#52514e"))
    for i, x in enumerate(v.values): a.text(ci[i, 1] + .2, i, f"{x:.2f}",
va="center", fontsize=8)
    a.set_title(f"{VN[mode]}: Vol ngoài mẫu %/năm (gross), CI 95% block
bootstrap b=21", fontsize=10); a.grid(axis="x", alpha=.3)
ax[0].set_xlim(0, None); plt.tight_layout(); savefig(fig, "T3_benchmark")

pc = [c for c in T3C.columns if c.startswith("p ")]
M = T3C.set_index(["mode", "pair"])[pc]
fig, a = plt.subplots(figsize=(11, 4.2))
im = a.imshow(-np.log10(M.values.clip(1e-4)), cmap="Purples", vmin=0, vmax=4,
aspect="auto")
a.set_xticks(range(len(pc)), [c[2:] for c in pc], rotation=30, ha="right",
fontsize=8)
a.set_yticks(range(len(M)), [f"{VN[m]} | {p}" for m, p in M.index],
fontsize=8)
for (y, x), p in np.ndenumerate(M.values): a.text(x, y, f"{p:.3f}",
ha="center", va="center", fontsize=8,
                                                  color="white" if p < 0.01
else "black")
plt.colorbar(im, ax=a, label="−log10 p"); a.set_title("T3 — p-value hai phía
(LW 2011 phương sai; JKM Sharpe; Holm trên 5 so sánh/chế độ)", fontsize=10)
plt.tight_layout(); savefig(fig, "T3_pvalue_heatmap")
```





Nhận xét T3d.

- **Hình trên.** CI 95% của Vol từng phương pháp (≈ ±1,5 pp) chồng lấn mạnh giữa các bộ co rút. Nhìn thanh lỗi đơn lẻ thì không phân biệt được EW-RIE*, EW-WeSpeR, EW-NL-LW, NL-LW, PCA. Chỉ 1/N tách hẳn.
- **Heatmap p-value.** Kiểm định *cặp* phân biệt được vì các chuỗi lợi suất tương quan rất cao, nên chênh lệch đo chính xác hơn nhiều.
  - Các ô tím đậm ($p < 0,01$): EW-RIE* với NL-LW, EW-NL-LW, PCA-auto, 1/N.
  - Hai hàng **EW-RIE* vs EW-WeSpeR** trắng (p 0,47–0,84): hai bộ ước lượng tương đương về rủi ro.
  - Mọi cột JKM (Sharpe) đều nhạt.

T4 – Lý thuyết: chọn nhánh nghiệm, điều kiện ngoại lai, hiệu chỉnh N hữu hạn, tính mới

Câu hỏi: các chi tiết kỹ thuật của EW-RIE* có cơ sở lý thuyết không? Mỗi mục theo thứ tự *phát biểu → chứng minh (hoặc ý chứng minh) → kiểm chứng số* (T4a–T4c). Mục cuối rà soát tính mới. Phần nào **đã chứng minh**, phần nào **chỉ kiểm chứng số** được ghi rõ.

Ký hiệu. $E = \sum_t w_t x_t x_t^\top = C^{1/2} W C^{1/2}$, $\sum_t w_t = 1$, $W = \sum_t w_t y_t y_t^\top$ (nhiễu), $a_t = N w_t$, $g_E(z) = \frac{1}{N}\mathrm{tr}(z - E)^{-1}$, $t_E(z) = z g_E(z) - 1$, $R_W(g) = \sum_t \dfrac{w_t}{1 - a_t g}$, $q_{\text{eff}} = N \sum_t w_t^2 = N/T_{\text{eff}}$. Công thức làm sạch (BBP 2016): $\xi_i = \lambda_i \dfrac{\mathrm{Im}\,\gamma(z_i)}{\mathrm{Im}\,t_E(z_i)}$, $z_i = \lambda_i - i\eta$, $\gamma R_W(\gamma) = t_E$.

(1) Trọng số đều (Mệnh đề 1 – đã chứng minh). Nếu $w_t = 1/T$ thì $R_W(g) = 1/(1 - qg)$, $q = N/T$; phương trình $\gamma/(1 - q\gamma) = t$ cho $\gamma = t/(1 + qt)$. Khi đó $\mathrm{Im}\,\gamma = \mathrm{Im}\,t/|1 + qt|^2$ và $1 + qt = 1 - q + qzg$, nên $\xi = \lambda/|1 - q + qzg|^2$: đúng RIE cổ điển. (Kiểm chứng số ở Test 1 của thầy: sai lệch $3 \cdot 10^{-15}$.)

(2) Chọn nhánh (Mệnh đề 2). Đặt $F(\gamma) = \gamma R_W(\gamma)$. Quy đồng thì $F(\gamma) = t$ là phương trình đa thức bậc T nên có T nghiệm; khi $t \to 0$ chúng tiến về $\gamma = 0$ và về $T - 1$ không điểm của R_W (nằm giữa các cực $1/a_t$). Vì $F(0) = 0$, $F'(0) = \sum_t w_t = 1$, theo định lý hàm ẩn tồn tại **duy nhất** một nghiệm giải tích $\gamma(t) = t - q_{\text{eff}} t^2 + O(t^3)$ gần 0. Nghiệm vật lý là nghiệm này vì khi $|z| \to \infty$, $t_E(z) = O(1/z) \to 0$ và khai triển moment đòi hỏi $\gamma \sim t$ (nhánh đi qua 0). Tiếp tục giải tích dọc tia $s\,t_E$, $s : 0 \to 1$ là đơn trị **miễn là** $F'(\gamma(s t_E)) \neq 0$ trên cả tia (không gặp điểm rẽ nhánh). *Đã chứng minh:* tồn tại và duy nhất ở lân cận 0; *chỉ kiểm chứng số:* $F' \neq 0$ dọc tia với trọng số mũ thực tế (T4a). Chứng minh tổng quát điều kiện này là việc còn mở.

(3) Ngoại lai (Mệnh đề 3). Với $C = I + (\theta - 1)vv^\top$: E cùng phổ với $W^{1/2} C W^{1/2} = W + (\theta - 1)uu^\top$, $u = W^{1/2} v$. Một trị riêng λ nằm ngoài phổ của W là ngoại lai khi $1 = (\theta - 1)\, v^\top W(\lambda - W)^{-1} v$, và khi $N \to \infty$ vế phải $\to (\theta - 1)\, t_W(\lambda)$ với $t_W = z g_W - 1$. Vì t_W giảm theo $\lambda > \lambda_+$ (biên trên của phổ W), ngoại lai tồn tại **khi và chỉ khi** $(\theta - 1) t_W(\lambda_+) > 1$, tức $\theta > \theta_c := 1 + 1/t_W(\lambda_+)$. Trọng số đều: $t_W(\lambda_+) = 1/\sqrt{q} \Rightarrow \theta_c = 1 + \sqrt{q}$ (ngưỡng BBP). Với trọng số mũ, đặt m là Stieltjes của W, $r(m) = \sum_t w_t/(1 + a_t m)$: $z(m) = r(m) - 1/m$, $t_W = -m\, r(m)$, $\lambda_+ = \min_m z(m)$; từ đó tính được θ_c và vị trí $\lambda^*(\theta)$ bằng số. *Công thức ξ cho ngoại lai.* Ở trị riêng cô lập, $\mathrm{Im}\,t_E \to 0$ khi $\eta \to 0$ nên tỷ số $\mathrm{Im}\,\gamma / \mathrm{Im}\,t$ là dạng $0/0$; vì γ giải tích theo t, tỷ số $\to \gamma'(t)$ với t thực, tức $\xi = \lambda \gamma'(t)$, với $g_E(\lambda_i) \approx \frac{1}{N} \sum_{j \neq i} (\lambda_i - \lambda_j)^{-1}$ (bỏ-một). Công thức phức với $\eta = N^{-1/2}$ cố định làm trơn quá mức một trị riêng tách xa bulk nên thiên lệch. *Đã chứng minh (ở mức vật lý):* điều kiện ngoại lai và ngưỡng; *kiểm chứng số:* vị trí λ_1 và độ lệch của công thức phức (T4b).

(4) Hiệu chỉnh N hữu hạn (heuristic, không có chứng minh). Khi $C = I$ thì $\xi_i \equiv 1$, nhưng công thức áp lên phổ mẫu hữu hạn (với $\eta > 0$) cho $\hat{\xi}_{\text{null}}(\lambda) \neq 1$. EW-RIE* chia ξ của bulk cho $\hat{\xi}_{\text{null}}$ (cùng công thức trên phổ nhiễu thuần mô phỏng, khớp thang theo trung vị), chỉ nhân lên khi $\hat{\xi}_{\text{null}} < 1$. T4c đo hiệu quả trên nhiễu thuần.

In [31]:
```
# ---- T4a. Chọn nhánh nghiệm cử a γ·R_W(γ) = t (kiểm chứng số´ trên một kỳ
thật giữa mã˜u) ----
Wn = CONFIG["WINDOW"]; u4 = U0[len(U0) // 2]
X4 = R_BT.iloc[POS[u4["date"]] - Wn:POS[u4["date"]]]
[u4["tickers"]].fillna(0).values
w4 = ew_weights(len(X4), CONFIG["HL_EW"]); C4, _ = ew_corr(X4, w4); lam4 =
np.linalg.eigvalsh(C4); N4 = len(lam4); a4 = N4 * w4
eta4 = 1 / np.sqrt(N4); z4 = lam4 - 1j * eta4; t4 = z4 * (1 / (z4[:, None] -
lam4[None, :])).mean(1) - 1
g4 = 1e-3 * t4; dFmin = np.full(N4, np.inf)                  # tiế´p tục
giả i tích theo tia s·t, theo dõi |F'(γ)| (F = γ·R_W)
for s in np.geomspace(1e-3, 1.0, 60):
    g4, res4 = _solve_gamma(s * t4, a4, w4, g4); _, R4, dR4 = _F(g4, s * t4,
a4, w4); dFmin = np.minimum(dFmin, np.abs(R4 + g4 * dR4))
assert np.allclose(g4, _continuation(t4, a4, w4))            # trùng với hàm
dùng trong EW-RIE*
phys = (lam4 * g4.imag / t4.imag)
# (a) nhiề u nghiệm: Newton từ 400 điể m xuấ´t phát ngã˜u nhiên cho cùng
phương trình
rng4 = np.random.default_rng(0); rows = []
for i in (5, 25, 50, 75, N4 - 2, N4 - 1):
    g0 = (rng4.standard_normal(400) + 1j * rng4.standard_normal(400)) *
np.abs(t4[i]) * 10 ** rng4.uniform(-1, 2, 400)
    gs, rs = _solve_gamma(np.full(400, t4[i]), a4, w4, g0, iters=100); ok =
np.isfinite(gs) & (rs < 1e-10); gs = gs[ok]
    hit = np.abs(gs - g4[i]) < 1e-6 * abs(g4[i]) + 1e-12; xi_w = (lam4[i] *
gs.imag / t4[i].imag)[~hit]
    rows.append({"trị riêng thứ": f"{i + 1}/{N4}", "λ": lam4[i], "số´xuất
phát hội tụ": int(ok.sum()), "số´nghiệm khác nhau":
len(np.unique(np.round(gs, 6))),
                 "% về nghiệm vật lý": 100 * hit.mean(), "ξ nghiệm sai / ξ
vật lý (trung vị)": np.median(xi_w) / phys[i] if len(xi_w) else np.nan, "ξ (z
= λ − iη, η = N^-½)": phys[i]})
T4A = pd.DataFrame(rows).set_index("trị riêng thứ")
print(f"Kỳ {u4['date'].date()} (N={N4}, T={Wn}, h={CONFIG['HL_EW']}, T_eff={1
/ (w4**2).sum():.0f}, q_eff={N4 * (w4**2).sum():.2f})")
print(f"Nhánh liên tục hóa: phần dư max = {res4.max():.1e} | Im γ/Im t > 0 ở
{int((g4.imag / t4.imag > 0).sum())}/{N4} trị riêng | min |F'(γ)| dọc tia =
{dFmin.min():.3f} (F'(0) = 1)")
display(T4A.round(3))
```

```
Kỳ 2021-11-24 (N=100, T=250, h=60, T_eff=155, q_eff=0.65)
Nhánh liên tục hóa: phần dư max = 2.1e-14 | Im γ/Im t > 0 ở 100/100 trị riêng |
min |F'(γ)| dọc tia = 0.317 (F'(0) = 1)
```

trị riêng thứ	λ	số xuất phát hội tụ	số nghiệm khác nhau	% về nghiệm vật lý	ξ nghiệm sai / ξ vật lý (trung vị)	ξ (z = λ − iη, η = N^-½)
6/100	0.063	393	1	100.000	NaN	0.192
26/100	0.157	394	2	99.746	0.000	0.388
51/100	0.371	390	1	100.000	NaN	0.552
76/100	0.888	393	1	100.000	NaN	0.787
99/100	8.544	396	3	99.495	0.004	6.183
100/100	29.565	381	87	0.525	1.000	0.088

Nhận xét T4a (chọn nhánh). Trên một kỳ thật (24/11/2021, N = 100, T_eff = 155):

- **Tiếp tục giải tích hoạt động đúng:** phần dư max $\mathbf{2 \cdot 10^{-14}}$; $\mathbf{\operatorname{Im}\gamma / \operatorname{Im} t > 0}$ ở 100/100 trị riêng; $\mathbf{\min |F'(\gamma)| = 0{,}317}$ dọc mọi tia (so với $\boldsymbol{F'(0) = 1}$) — không gặp điểm rẽ nhánh, nên điều kiện của Mệnh đề 2 được thỏa trong số liệu thật (chỉ kiểm chứng số, chưa chứng minh tổng quát).
- **Newton "mù" từ điểm ngẫu nhiên:** ở trị riêng bulk, 99,5–100% điểm xuất phát vẫn về nghiệm vật lý (1–3 nghiệm khác nhau), nhưng nghiệm sai cho $\boldsymbol{\xi \approx 0}$ (tỷ số 0,000–0,004) ⇒ ma trận gần suy biến. Với **trị riêng cô lập** (mode thị trường, λ ≈ 29,6) chỉ **0,5%** điểm xuất phát về đúng nghiệm, 87 nghiệm khác nhau ⇒ phải dùng tiếp tục giải tích chứ không dùng Newton trực tiếp.
- Cột cuối: công thức phức với $\boldsymbol{\eta = N^{-1/2}}$ cho mode thị trường $\boldsymbol{\xi = 0{,}09}$ trong khi λ = 29,6 — sai nặng; xem T4b.

In [32]:
```
# ---- T4b. Điều kiện ngoại lai (mode thị trường): ngưỡng tách khỏi bulk, vị trí trị riêng và công thức ξ ----
# Mô hình "spike": C = I + (θ−1) v vᵀ, E = Σ w_t x_t x_tᵀ. Điều kiện ngoại lai (θ−1)·t_W(λ) = 1, với t_W = zg_W − 1 của nhiễu W.
# Với m(z) = Stieltjes của W (m = 1/(r(m) − z), r(m) = Σ w_t/(1 + N w_t m)): z(m) = r(m) − 1/m, t_W = −m·r(m); biên trên λ₊ = min_m z(m).
from scipy.optimize import minimize_scalar, brentq
def spike_theory(w, N, theta):
    r = lambda m: (w / (1 + N * w * m)).sum(); mmin = -1 / (N * w.max())
    ms = minimize_scalar(lambda m: r(m) - 1 / m, bounds=(mmin * (1 - 1e-6), -1e-6), method="bounded", options=dict(xatol=1e-14)).x
    t_edge = -ms * r(ms); theta_c = 1 + 1 / t_edge
    if theta <= theta_c: return theta_c, np.nan
    m = brentq(lambda m: -m * r(m) - 1 / (theta - 1), ms, -1e-12); return theta_c, r(m) - 1 / m
_q = 0.4; _wu = np.full(250, 1 / 250)                                   # kiểm tra: trọng số đều ⇒ θ_c = 1 + √q (BBP)
assert abs(spike_theory(_wu, 100, 5.0)[0] - (1 + np.sqrt(_q))) < 1e-4
Nn, Tn, hln = 100, 250, CONFIG["HL_EW"]; wn = ew_weights(Tn, hln); rngs = np.random.default_rng(1); rows = []
for theta in (1.5, 2.0, 3.0, 5.0, 10.0):
    th_c, lam_pred = spike_theory(wn, Nn, theta); l1, orc, xo, xb = [], [], [], []
    for _ in range(30):
        v = rngs.standard_normal(Nn); v /= np.linalg.norm(v); A = np.eye(Nn) + (np.sqrt(theta) - 1) * np.outer(v, v)
        Xs = rngs.standard_normal((Tn, Nn)) @ A; Es = (Xs * wn[:, None]).T @ Xs; ls, Us = np.linalg.eigh(Es); Cs = A @ A
        l1.append(ls[-1]); orc.append(Us[:, -1] @ Cs @ Us[:, -1])
        xo.append(_xi_outliers(ls, [Nn - 1], wn, Nn)[0]); xb.append(_xi(ls, ls[-1:], wn, Nn, 1 / np.sqrt(Nn))[0])
    rows.append({"θ": theta, "ngưỡng θ_c": th_c, "λ₁ lý thuyết": lam_pred, "λ₁ mô phỏng": np.mean(l1), "ξ oracle (ûᵀCû)": np.mean(orc),
                 "ξ EW-RIE* (trục thực)": np.mean(xo), "ξ BBP gốc (η = N^-½)": np.mean(xb)})
T4B = pd.DataFrame(rows).set_index("θ")
print(f"Spike: N={Nn}, T={Tn}, h={hln} (T_eff={1 / (wn**2).sum():.0f}); TB 30 lần mô phỏng. Kiểm tra trọng số đều: θ_c = 1+√q ✔")
display(T4B.round(3))
```

```
Spike: N=100, T=250, h=60 (T_eff=155); TB 30 lần mô phỏng. Kiểm tra trọng số đều: θ_c = 1+√q ✔
```

θ	ngưỡng θ_c	λ₁ lý thuyết	λ₁ mô phỏng	ξ oracle (ûᵀCû)	ξ EW-RIE* (trục thực)	ξ BBP gốc (η = N^-½)
1.5	1.983	NaN	3.416	1.039	1.219	1.227
2.0	1.983	3.509	3.470	1.146	1.273	1.281
3.0	1.983	4.052	4.053	2.163	2.209	2.080
5.0	1.983	5.842	5.673	4.249	4.139	3.711
10.0	1.983	10.731	10.573	9.334	9.284	6.512

Nhận xét T4b (ngoại lai). Mô hình spike, N = 100, T = 250, h = 60 (T_eff = 155), trung bình 30 lần mô phỏng:

- **Ngưỡng:** θ_c = 1,98 với trọng số mũ, cao hơn ngưỡng BBP 1+√q = 1,63 của trọng số đều cùng N/T (kiểm chứng assert: trọng số đều cho đúng 1+√q). Trọng số mũ tập trung thông tin vào ít quan sát nên spike khó tách hơn. θ = 1,5 dưới ngưỡng: không dự đoán ngoại lai, λ_1 mô phỏng 3,42 là biên bulk.
- **Vị trí:** λ_1 lý thuyết khớp mô phỏng trong ≤ 3% (θ = 3: 4,052 so với 4,053; θ = 10: 10,73 so với 10,57).
- **ξ của ngoại lai** (so với oracle $\hat{u}^T C \hat{u}$): EW-RIE* trên trục thực lệch +2,1% (θ = 3), −2,6% (θ = 5), −0,5% (θ = 10); công thức BBP gốc (η = N^-½) lệch −3,8%, −12,7% và **−30%**. Với mode thị trường lớn công thức phức đánh giá thấp nặng ⇒ xử lý riêng ngoại lai là cần thiết.
- **Lưu ý:** hai dòng θ ≤ 2 gần/dưới ngưỡng, trị riêng chưa tách khỏi bulk nên công thức cô lập lệch +0,1–0,2 (cả hai bộ ước lượng); trong EW-RIE* thật các trị riêng đó đi nhánh bulk.

In [33]:
```
# ---- T4c. Hiệu chỉnh N hữu hạn: dữ liệu nhiễu thuần (C = I) ⇒ ξ đúng phải bằng 1 ⇒ danh mục tối ưu là 1/N ----
# Chỉ số: N·‖w‖² (= rủi ro GMV / rủi ro tối ưu khi C = I; 1 = hoàn hảo). Trung bình 20 lần mô phỏng, N = 100, T = 250.
rngc = np.random.default_rng(2); rows = []
for hl_ in (30, 60):
    wc = ew_weights(250, hl_); acc = {}
    for _ in range(20):
        Cc, _ = ew_corr(rngc.standard_normal((250, 100)), wc)
        for k_, Ch in {"EW-Sample": Cc, "EW-NL-LW": renorm(nl_lw_S(Cc, 1 / (wc**2).sum())), "EW-RIE (q_eff)": rie_corr(Cc, 100 * (wc**2).sum()),
                       "EW-RIE* (không hiệu chỉnh N hữu hạn)": ew_rie_corr(Cc, wc, debias=False), "EW-RIE* (có hiệu chỉnh)": ew_rie_corr(Cc, wc),
                       "EW-WeSpeR": ew_wesper_corr(Cc, wc)}.items():
            wh = gmv(Ch); acc.setdefault(k_, []).append(100 * wh @ wh)
    rows += [dict(h=hl_, T_eff=1 / (wc**2).sum(), method=k_, **{"N·‖w‖²": np.mean(v)}) for k_, v in acc.items()]
T4C = pd.DataFrame(rows).pivot(index="method", columns=["h"], values="N·‖w‖²")
print("Rủi ro GMV / rủi ro tối ưu trên nhiễu thuần (1 = hoàn hảo); T_eff:", {h_: round(1 / (ew_weights(250, h_)**2).sum()) for h_ in (30, 60)});
display(T4C.round(3))
```

```
Rủi ro GMV / rủi ro tối ưu trên nhiễu thuần (1 = hoàn hảo); T_eff: {30: 86, 60: 155}
```

h	30	60
method		
EW-NL-LW	1.431	1.041
EW-RIE (q_eff)	1.095	1.028
EW-RIE* (có hiệu chỉnh)	1.058	1.006
EW-RIE* (không hiệu chỉnh N hữu hạn)	1.176	1.058
EW-Sample	2.522	1.864
EW-WeSpeR	1.120	1.000

Nhận xét T4c (N hữu hạn). Trên nhiễu thuần (C = I), danh mục tối ưu là 1/N nên chỉ số $N\cdot\|w\|^2 = 1$ là hoàn hảo:

- **h = 60** (T_eff = 155): EW-RIE* có hiệu chỉnh **1,006**, không hiệu chỉnh 1,058 ⇒ hiệu chỉnh giảm ~90% phần rủi ro dư. Các bộ khác: EW-WeSpeR 1,000, EW-RIE (q_eff) 1,028, EW-NL-LW 1,041, EW-Sample 1,864.
- **h = 30** (T_eff = 86 < N): có hiệu chỉnh **1,058**, không hiệu chỉnh 1,176 (giảm ~67%); EW-WeSpeR 1,120, EW-RIE (q_eff) 1,095, EW-NL-LW 1,431, EW-Sample 2,52.
- Hiệu chỉnh N hữu hạn (heuristic) có tác dụng rõ trên nhiễu thuần và không làm xấu đi kết quả; nhưng đây chưa phải chứng minh cho dữ liệu có cấu trúc (Test 2b).

Rà soát tính mới — chưa đọc toàn văn nên cần nhóm xác nhận trước khi viết bài (phiên chạy không truy cập được arXiv/GitHub; chỉ dựa vào tóm tắt bài và ghi chú của nhóm).

Thành phần	Đã có trong tài liệu?	Ghi chú
Công thức RIE cho nhiễu nhân tổng quát $\xi = \lambda \operatorname{Im}\gamma / \operatorname{Im} t_E$ với R-transform của nhiễu	**Có** (ghi chú của nhóm)	Bun–Bouchaud–Potters 2016, 2017; cần đọc lại §III để trích đúng
Co phi tuyến cho hiệp phương sai mẫu **có trọng số**	**Có**	Oriol 2024 (WeSpeR, arXiv:2410.14413): mở rộng QuEST cho trọng số; khôi phục phổ quần thể
Trường hợp trọng số mũ ($a_t = N w_t$)	Là trường hợp riêng của công thức tổng quát	
Chọn nhánh bằng tiếp tục giải tích dọc tia (T4a)	**Chưa thấy**	cần tìm trong 3 tài liệu trên
Ngoại lai trên trục thực, $\xi = \lambda\gamma'(t)$, g bỏ-một (T4b)	**Chưa thấy**	RIE gốc dùng $\eta = N^{-1/2}$
Hiệu chỉnh N hữu hạn bằng phổ nhiễu thuần (T4c)	**Chưa thấy**	BBP 2017 có hiệu chỉnh khác cho trọng số đều (theo chú thích mã của thầy)

Bằng chứng thực nghiệm (từ T2–T4): EW-RIE* ≈ EW-WeSpeR về rủi ro ngoài mẫu (T3c: p ≥ 0,47); hơn EW-NL-LW rất ít (0,06–0,16 pp); kém ổn định khi q_eff > 2 (Test 2b: 1,72 so với WeSpeR 1,39). Tốc độ: EW-RIE* ≈ 0,17 s/lần; WeSpeR bản cài lại ở đây 1–4 s/lần (đo ở `EW_RIE_HOSE_v3.ipynb`, T6; chưa tối ưu như bản gốc dùng PyTorch). **Gợi ý định vị (ý kiến của tui, nhóm quyết định):** không tuyên bố vượt WeSpeR về độ chính xác; luận điểm khả dĩ là công thức đóng, nhanh, kèm các chi tiết triển khai (nhánh, ngoại lai, N hữu hạn) và giới hạn rõ ràng ở q_eff > 2.

T5 – Vì sao VS giúp trên HOSE?

Câu hỏi: chuẩn hóa biến động (VS) có giảm rủi ro không, khi nào, và nhờ cơ chế nào?

- **T5a** ablation chỉ đổi VS: RIE ↔ VS-RIE, NL-LW ↔ VS-NL-LW; theo giai đoạn thị trường.
- **T5b** dự báo biến động 21 phiên: EWMA vs cửa sổ phẳng (RMSE, QLIKE) theo giai đoạn.
- **T5c** phổ trị riêng trước/sau VS và condition number của Σ đưa vào GMV.
- VS-EW-RIE* **không** làm ở đây (chưa có biến thể đã kiểm thử; không nhầm với VS-RIE).

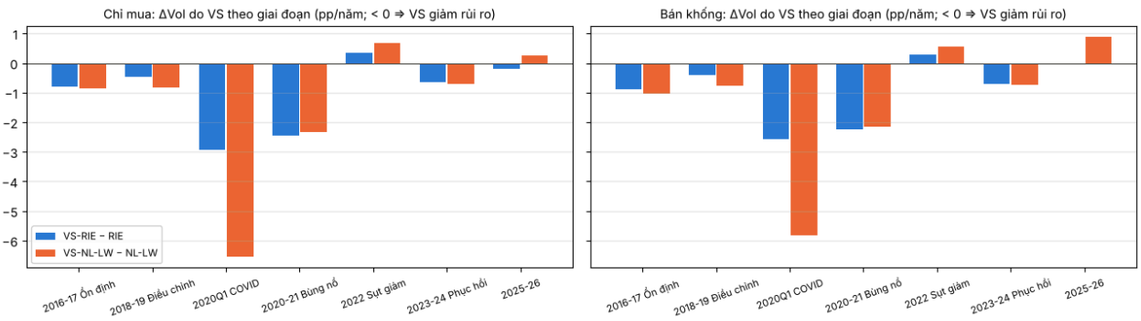
In [34]:
```
# ---- T5: Vì sao VS giúp trên HOSE? ----
# T5a. Ablation: mỗi cặp chỉ khác bước chuẩn hóa biến động (VS): RIE ↔ VS-RIE, NL-LW ↔ VS-NL-LW. Dùng lại lần chạy chính §4 (QUICK=False).
ABL = [("RIE", "VS-RIE"), ("NL-LW", "VS-NL-LW")]
rows, per = [], []
for m in MODES:
    out = RESULTS[m]["out"]
    for b, v in ABL:
        lo_, hi_ = block_bootstrap_ci(out[v]["r"].values, out[b]["r"].values)
        mb, mv = metrics(out[b]), metrics(out[v])
        rows.append({"Chế độ": VN[m], "Cặp": f"{v} vs {b}", "Vol gốc":
mb["Vol %"], "Vol VS": mv["Vol %"], "ΔVol (pp)": mv["Vol %"] - mb["Vol %"],
                     "CI lo": lo_, "CI hi": hi_, "Sharpe sau phí gốc":
mb["Sharpe sau phí"], "Sharpe sau phí VS": mv["Sharpe sau phí"],
                     "Turnover gốc": mb["Turnover/kỳ"], "Turnover VS":
mv["Turnover/kỳ"]})
        per.append((RESULTS[m]["periods"].loc[v] - RESULTS[m]
["periods"].loc[b]).rename((VN[m], f"{v} − {b}")))
T5A = pd.DataFrame(rows).set_index(["Chế độ", "Cặp"]); T5P =
pd.DataFrame(per).T
save(T5A, "T5a_vs_ablation"); save(T5P, "T5a_vs_ablation_periods")
print("Bảng T5a — tác động của VS (Vol %/năm gộp; CI 95% block bootstrap b=21 của ΔVol)"); show(T5A)
print("ΔVol (pp) do VS theo giai đoạn thị trường (< 0: VS giảm rủi ro):");
show(T5P, 2)
fig, ax = plt.subplots(1, 2, figsize=(13, 3.8), sharey=True)
for a, m in zip(ax, VN.values()):
    d = T5P[m]; x = np.arange(len(d))
    for j, (c, col) in enumerate(zip(d.columns, ["#2a78d6", "#eb6834"])):
a.bar(x + (j - .5) * .38, d[c], .36, color=col, label=c)
    a.axhline(0, color="#52514e", lw=.8); a.set_xticks(x, d.index,
rotation=20, fontsize=8); a.grid(axis="y", alpha=.3)
    a.set_title(f"{m}: ΔVol do VS theo giai đoạn (pp/năm; < 0 ⇒ VS giảm rủi ro)", fontsize=10)
ax[0].legend(fontsize=8); plt.tight_layout(); savefig(fig, "T5a_vs_periods")
pm = T5P.mean(axis=1); tr = T5A["Turnover VS"] / T5A["Turnover gốc"]
```

```
Bảng T5a — tác động của VS (Vol %/năm gộp; CI 95% block bootstrap b=21 của ΔVol)
```

Chế độ	Cặp	Vol gốc	Vol VS	ΔVol (pp)	CI lo	CI hi	Sharpe sau phí gốc	Sharpe sau phí VS	Turnover gốc	Turnover VS
Chỉ mua	VS-RIE vs RIE	14,016	13,298	-0,718	-1,320	-0,209	0,243	0,203	0,350	0,682
	VS-NL-LW vs NL-LW	13,721	12,933	-0,788	-1,481	-0,119	0,077	0,156	0,343	0,745
Bán khống	VS-RIE vs RIE	13,299	12,634	-0,666	-1,262	-0,162	0,228	0,095	0,554	0,992
	VS-NL-LW vs NL-LW	13,100	12,487	-0,614	-1,277	0,061	-0,051	-0,083	0,812	1,408

ΔVol (pp) do VS theo giai đoạn thị trường (< 0: VS giảm rủi ro):

	Chỉ mua		Bán khống	
	VS-RIE − RIE	VS-NL-LW − NL-LW	VS-RIE − RIE	VS-NL-LW − NL-LW
2016-17 Ổn định	-0,80	-0,86	-0,89	-1,05
2018-19 Điều chỉnh	-0,45	-0,82	-0,39	-0,76
2020Q1 COVID	-2,92	-6,54	-2,58	-5,81
2020-21 Bùng nổ	-2,46	-2,32	-2,25	-2,16
2022 Sụt giảm	0,36	0,67	0,28	0,55
2023-24 Phục hồi	-0,64	-0,71	-0,72	-0,75
2025-26	-0,19	0,27	-0,04	0,89



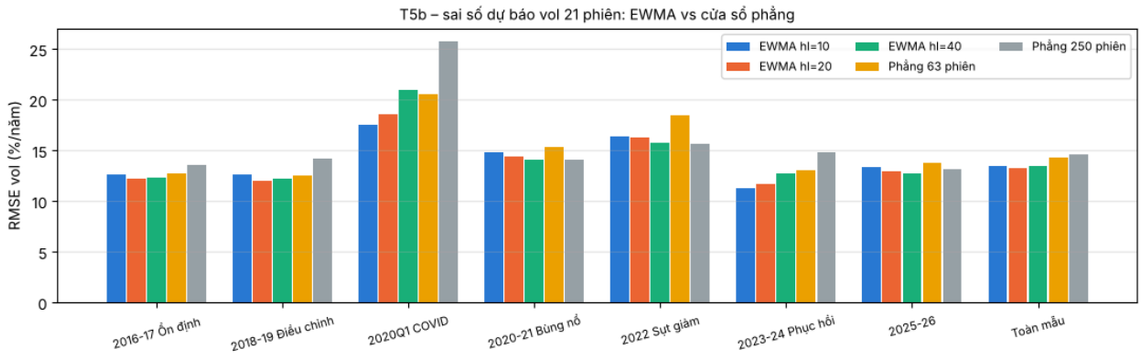
Nhận xét T5a.

- **Tác động tổng:** VS giảm Vol ở **4/4 cặp**, từ −0,61 đến −0,79 pp/năm. 3/4 cặp có CI 95% nằm hẳn dưới 0; riêng VS-NL-LW vs NL-LW bán khống có CI [−1,28; +0,06].
- **Theo giai đoạn** (hình):
  - Lợi ích tập trung ở **COVID 2020Q1** (−2,6 đến −6,5 pp) và **2020–21** (≈ −2,2 đến −2,5 pp).
  - VS **làm Vol tăng năm 2022** (+0,3 đến +0,7 pp), cả 4 cặp, và với NL-LW năm 2025–26 (+0,3/+0,9 pp).
- **Cái giá:** turnover tăng gấp ≈ 1,7–2,2 lần, nên Sharpe sau phí giảm ở 3/4 cặp.
- **Liên hệ §4:** VS-NL-LW thấp nhất toàn mẫu ở v3, nhưng lợi thế đó dựa phần lớn vào một giai đoạn (COVID–2021).

In [35]:
```
# T5b. Dự báo phương sai 21 phiên tới: EWMA (hl ∈ {10, 20, 40}, đúng hàm
ewma_standardize mà VS dùng) vs cửa sổ phẳng 63/250 phiên.
# Mẫu: các mã trong universe chuẩn ở mỗi kỳ; dự báo chỉ dùng cửa sổ
quá khứ. Mục tiêu RV = trung bình r² của 21 phiên sau.
# Sai số: RMSE của vol năm hóa (%) và QLIKE = log h + RV/h (dạng Patton
2011, xác định cả khi RV = 0).
W, H = CONFIG["WINDOW"], 21; rows = []
for u in U0:
    s = POS[u["date"]]
    if s + H > len(R_BT): continue
    X = R_BT.iloc[s - W:s][u["tickers"]].fillna(0).values; rv =
(R_BT.iloc[s:s + H][u["tickers"]].fillna(0).values ** 2).mean(0)
    fc = {**{f"EWMA hl={h}": ewma_standardize(X, h)[1] ** 2 for h in (10, 20,
40)}, **{f"Phẳng {w} phiên": (X[-w:] ** 2).mean(0) for w in (63, W)}}
    per_ = next((p for p, a_, b_ in PERIODS if pd.Timestamp(a_) <= u["date"]
<= pd.Timestamp(b_)), None)
    for k, h in fc.items():
        h = np.maximum(h, 1e-12); rows.append(dict(period=per_, model=k,
se=np.mean((np.sqrt(h) - np.sqrt(rv)) ** 2), ql=np.mean(np.log(h) + rv / h)))
F = pd.DataFrame(rows); F = pd.concat([F, F.assign(period="Toàn mẫu")])
T5B = F.groupby(["period", "model"], sort=False).agg(RMSE=("se", lambda x:
np.sqrt(x.mean() * 252) * 100), QLIKE=("ql", "mean"), n=("se", "size"))
save(T5B, "T5b_vol_forecast")
tbl = T5B["RMSE"].unstack()[["EWMA hl=10", "EWMA hl=20", "EWMA hl=40", "Phẳng
63 phiên", f"Phẳng {W} phiên"]]
tbl.loc["QLIKE (toàn mẫu)"] = T5B.xs("Toàn mẫu")["QLIKE"]
print("Bảng T5b — RMSE dự báo vol 21 phiên (%/năm) theo giai đoạn; dòng cuối:
QLIKE toàn mẫu (thấp hơn = tốt hơn)"); show(tbl)
fig, ax = plt.subplots(figsize=(11, 3.6)); M = tbl.drop(index="QLIKE (toàn
mẫu)"); x = np.arange(len(M))
for j, (c, col) in enumerate(zip(M.columns, ["#2a78d6", "#eb6834", "#1baf7a",
"#eda100", "#9aa0a6"])): ax.bar(x + (j - 2) * .16, M[c], .15, color=col,
label=c)
ax.set_xticks(x, M.index, rotation=15, fontsize=8); ax.set_ylabel("RMSE vol
(%/năm)"); ax.legend(fontsize=8, ncol=3); ax.grid(axis="y", alpha=.3)
ax.set_title("T5b — sai số dự báo vol 21 phiên: EWMA vs cửa sổ phẳng",
fontsize=10); plt.tight_layout(); savefig(fig, "T5b_vol_forecast")
fa, cv = M.loc["Toàn mẫu"], M.loc["2020Q1 COVID"]
```

```
Bảng T5b — RMSE dự báo vol 21 phiên (%/năm) theo giai đoạn; dòng cuối: QLIKE to
àn mẫu (thấp hơn = tốt hơn)
```

model period	EWMA hl=10	EWMA hl=20	EWMA hl=40	Phẳng 63 phiên	Phẳng 250 phiên
2016-17 Ổn định	12,677	12,208	12,287	12,741	13,584
2018-19 Điều chỉnh	12,622	12,063	12,226	12,572	14,204
2020Q1 COVID	17,505	18,613	20,962	20,553	25,752
2020-21 Bùng nổ	14,849	14,359	14,075	15,366	14,114
2022 Sụt giảm	16,406	16,243	15,764	18,426	15,672
2023-24 Phục hồi	11,261	11,701	12,697	13,027	14,866
2025-26	13,360	12,912	12,732	13,809	13,145
Toàn mẫu	13,511	13,274	13,436	14,356	14,639
QLIKE (toàn mẫu)	-6,413	-6,464	-6,476	-6,416	-6,437



Nhận xét T5b. Dự báo vol 21 phiên trên toàn mẫu:

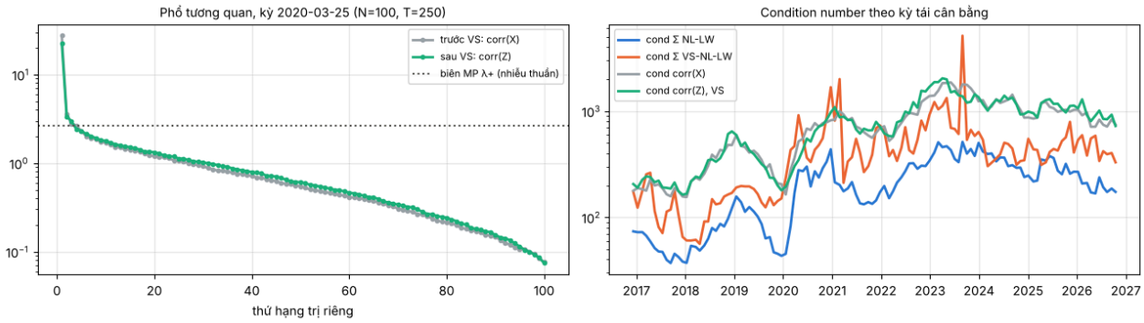
- RMSE tốt nhất là **EWMA hl = 20** (13,27%/năm), so với cửa sổ phẳng 250 phiên 14,64% và phẳng 63 phiên 14,36%.
- QLIKE tốt nhất là EWMA hl = 40.
- **Trong COVID 2020Q1** khoảng cách lớn nhất: EWMA hl = 10 cho 17,5% so với phẳng 250 phiên 25,8%.
- **Năm 2022** là giai đoạn duy nhất cửa sổ phẳng 250 phiên thắng (15,67% so với 15,76% của hl = 40). Năm này cũng là năm VS làm rủi ro tăng ở cả 4 cặp (T5a). Một giải thích có thể (chưa kiểm chứng): vol năm 2022 tăng theo nhiều đợt, nên EWMA "quên" nhanh và đánh giá thấp vol ở đợt giảm tiếp theo.

⇒ VS có lợi đúng khi biến động riêng lẻ đổi đột ngột và EWMA theo kịp. Điều này khớp giả thuyết cơ chế của T5.

In [36]:
```
# T5c. Phổ trị riêng trước/sau VS (1 kỳ ví dụ) + condition number theo kỳ.
"Trước": tương quan mẫu của X; "sau": của Z = X/σ_EWMA
# (bỏ HL_VOL phiên đầu, đúng make_vs). Thêm cond của hiệp phương sai THỰC
SỰ đưa vào GMV.
hv = CONFIG["HL_VOL"]; rows = []
winX = lambda u: R_BT.iloc[POS[u["date"]] - W:POS[u["date"]]]
[u["tickers"]].fillna(0).values
for u in U0:
    X = winX(u); Z = ewma_standardize(X, hv)[0][hv:]
    ex, ez = np.linalg.eigvalsh(np.corrcoef(X, rowvar=False)),
np.linalg.eigvalsh(np.corrcoef(Z, rowvar=False))
    r = {"date": u["date"], "cond corr(X)": ex[-1] / ex[0], "cond corr(Z),
VS": ez[-1] / ez[0], "λ1/N trước VS": ex[-1] / len(ex), "λ1/N sau VS": ez[-1]
/ len(ez)}
    for k in ("NL-LW", "VS-NL-LW", "RIE", "VS-RIE"): e =
check_cov(ESTIMATORS[k](X), k); r[f"cond Σ {k}"] = e[-1] / e[0]
    rows.append(r)
T5C = pd.DataFrame(rows).set_index("date"); save(T5C,
"T5c_condition_numbers")
print("Bảng T5c — condition number và trị riêng lớn nhất (trung bình / trung
vị trên các kỳ)"); show(T5C.agg(["mean", "median"]).T, 2)
ex_u = next(u for u in U0 if u["date"] >= pd.Timestamp("2020-03-01")); X =
winX(ex_u); Z = ewma_standardize(X, hv)[0][hv:]
fig, ax = plt.subplots(1, 2, figsize=(13, 3.8))
for lab, M_, c in [("trước VS: corr(X)", X, "#9aa0a6"), ("sau VS: corr(Z)",
Z, "#1baf7a")]:
    ev = np.sort(np.linalg.eigvalsh(np.corrcoef(M_, rowvar=False)))[::-1];
ax[0].plot(np.arange(1, len(ev) + 1), ev, color=c, lw=2, marker="o", ms=3,
label=lab)
ax[0].axhline((1 + np.sqrt(CONFIG["N"] / W)) ** 2, ls=":", color="#52514e",
label="biên MP λ+ (nhiễu thuần)")
ax[0].set_yscale("log"); ax[0].set_xlabel("thứ hạng trị riêng");
ax[0].legend(fontsize=8); ax[0].grid(alpha=.3)
ax[0].set_title(f"Phổ tương quan, kỳ {ex_u['date'].date()} (N={CONFIG['N']},
T={W})", fontsize=10)
for k, c in [("cond Σ NL-LW", "#2a78d6"), ("cond Σ VS-NL-LW", "#eb6834"),
("cond corr(X)", "#9aa0a6"), ("cond corr(Z), VS", "#1baf7a")]:
    ax[1].plot(T5C.index, T5C[k], color=c, lw=2, label=k)
ax[1].set_yscale("log"); ax[1].legend(fontsize=8); ax[1].grid(alpha=.3);
ax[1].set_title("Condition number theo kỳ tái cân bằng", fontsize=10)
plt.tight_layout(); savefig(fig, "T5c_spectrum_condition")
c = T5C.mean()
```

```
Bảng T5c — condition number và trị riêng lớn nhất (trung bình / trung vị trên c
ác kỳ)
```

	mean	median
cond corr(X)	766,03	730,61
cond corr(Z), VS	790,94	796,78
λ1/N trước VS	0,33	0,35
λ1/N sau VS	0,31	0,33
cond Σ NL-LW	212,13	199,13
cond Σ VS-NL-LW	465,25	389,28
cond Σ RIE	110,04	88,00
cond Σ VS-RIE	166,64	125,33



Nhận xét T5c.

- **Phổ tương quan:** trước và sau VS gần như trùng nhau (hình trái, kỳ 03/2020). Condition number TB 766 → 791; λ_1/N 0,33 → 0,31. Như vậy VS **không** làm sạch cấu trúc tương quan.
- **Ma trận Σ đưa vào GMV:** condition number còn **tăng** khi có VS (NL-LW 212 → VS-NL-LW 465; RIE 110 → VS-RIE 167). Lý do là biến động EWMA có độ phân tán giữa các mã lớn hơn.
- **Cơ chế:** lợi ích của VS (T5a) đến từ việc ước lượng *biến động riêng lẻ* theo kịp thay đổi chế độ, không từ tương quan sạch hơn. Kết hợp với T5b, điều này giải thích vì sao trong mô phỏng GARCH (vol trơn, không có cú sốc lớn) VS không giúp.
- **Hướng tiếp theo:** chưa làm VS-EW-RIE* (chuẩn hóa EWMA + EW-RIE* cho tương quan). Theo §4 và T5, đây là ứng viên tự nhiên để vượt cả VS-NL-LW.

Ghi chú

- **Khai báo dùng AI:** mã nguồn được xây dựng với hỗ trợ của trợ lý AI; nhóm cần rà soát, hiểu và khai báo theo quy định của tạp chí.
- **Tính mới:** công thức tổng quát cho nhiễu nhân có trong Bun–Bouchaud–Potters (2016, 2017). Cần T4 xác nhận phần cụ thể hóa cho EWMA, chọn nhánh, xử lý ngoại lai chưa được công bố.
- **Tái lập:** cố định `seed`; ghi lại phiên bản thư viện (ô đầu) vào phụ lục.

Ghi chú v3

- **Mã hủy niêm yết:** lợi suất các ngày sau ngày giao dịch cuối = 0 (giữ tiền mặt đến kỳ tái cân bằng kế tiếp), đúng quy ước của thầy; không áp mức phạt mất giá khi hủy bắt buộc. Rất hiếm khi xảy ra ngay trong kỳ nắm giữ.
- **19 mã đã hủy không lấy được giá điều chỉnh** từ VCI (`missing_tickers.csv`), chủ yếu hủy 2015–2018: survivorship bias chưa loại hoàn toàn.
- **EW-WeSpeR** là bản cài lại bằng numpy theo các phương trình của Oriol (2024); phiên chạy không truy cập được arXiv/GitHub của bài nên cần đối chiếu với mã gốc trước khi công bố.
- **EW-NL-LW (v3)** dùng nhánh p > n của Ledoit–Wolf (2020) khi N > T_eff với T_eff = 1/Σw² không nguyên: phép mở rộng hợp lý, không phải kết quả lý thuyết của LW cho trọng số mũ.
- **T6** (N, T, half-life) và VN30/HNX chưa làm trong bản gọn này (có trong `EW_RIE_HOSE_v3.ipynb`).
- **Chạy trên web (Colab):** chạy từ trên xuống; ô T1.0 tự tải `data_hose/` từ GitHub.